석사 학위논문

거대 언어 모델(LLM)과 생성형 3D Gaussian Splatting을 활용한 의미론적 3D 공간 자동 구성 시스템

A Semantic 3D Space Construction System Using Large Language Models and Generative 3D Gaussian Splatting

국 립 공 주 대 학 교 대 학 원

게임디자인학과

김 준 형

2026. 8.

석사 학위논문

거대 언어 모델(LLM)과 생성형 3D Gaussian Splatting을 활용한 의미론적 3D 공간 자동 구성 시스템

A Semantic 3D Space Construction System Using Large Language Models and Generative 3D Gaussian Splatting

지도교수 이 동 엽

이 논문을 석사학위 청구논문으로 제출함

2026. 5.

국 립 공 주 대 학 교 대 학 원
게임디자인학과
김 준 형

석사 학위논문

김 준 형 의 석사학위 청구논문을 인준함

2026. 6.

심 사 위 원 장　　김　준　협　　(인)

심 사 위 원　　이　동　엽　　(인)

심 사 위 원　　민　준　일　　(인)

국 립 공 주 대 학 교　대 학 원

국문 초록

인공지능의 관심이 텍스트·이미지·영상을 넘어 공간 생성으로 이동하면서, 거대 언어 모델과 생성형 3D 기법, 그리고 최근 부각된 월드 모델(world model)이 자연어로부터 3차원 장면을 합성하는 단계에 이르렀다. 다만 현재까지의 시스템은 장면을 "만들어내는" 단계에 무게가 쏠려 있으며, 생성된 장면이 기존 게임 엔진의 물리·논리 체계 안에서 실제로 작동하도록 만드는 층위는 상대적으로 덜 다뤄졌다. 본 논문의 Unity-SplatForge는 이 간극을 겨냥해, 생성형 3DGS 에셋과 LLM의 의미론적 배치 지시를 Unity 물리 엔진의 검증 루프 안에서 결합하는 하이브리드 저작 파이프라인을 제안한다.

파이프라인은 두 단계로 움직인다. 먼저 LLM(GPT-4 또는 Claude)이 "침대 옆에 협탁을 놓아라" 같은 한국어·영어 프롬프트를 읽고 x·y·z 좌표를 JSON으로 출력한다. 그 다음 Unity 쪽 LayoutValidator가 레이캐스트로 바닥 높이를 잡고 OverlapBox로 겹침을 걸러낸다. 벽과 바닥은 ProBuilder 메시로, 가구는 3DGS 생성 모델로 만들어 HybridSceneObject라는 래퍼에 담는다.

침실·사무실·거실 세 가지 방에 적용해 본 결과, 손작업 대비 작업 시간이 약 4/5가량 단축되었다. Phase 2 sweep(3 시나리오 × 3 조건 × N=5 × mock·openai 2 provider, 총 90 trial)에서 실 LLM(gpt-4o-mini, $0.0094 USD) 도입 시 침실·사무실의 Floor Adhesion이 14.29% → 42.86%로 약 3.0~3.4배 향상되었고, 충돌 횟수는 약 28~60% 감소(36~50건 → 20~30건)하면서 wall-clock 오버헤드는 +4~12%(30~103 ms/trial)에 머물렀다. LLM을 무작위 좌표 생성으로 대체하고 동일한 물리 보정만 돌리면 Semantic Proximity가 full 대비 약 75~80% 하락(예: 침실 0.79 → 0.19, 사무실 0.67 → 0.17)하여, 의미론적 추론 없이는 쓸 만한 방이 나오지 않음을 정량적으로 확증하였다.

본 연구는 추가로 macOS Apple Silicon 단일 기기에서 학습-임포트-렌더 전

구간 완결이 가능함을 PoC로 입증하였다. Brush(Rust+wgpu Metal, Apache-2.0) 기반 30K iter 학습이 Mip-NeRF360 bonsai에서 PSNR 32.21 dB(원 3DGS 32.4 dB와 0.19 dB 차)를 기록하였고, hloc + LightGlue로 SfM 단계를 1시간 46분에서 51분 6초로 2.07× 단축(종합 wall-clock 1.44× 절감)하면서 PSNR은 ±0.5 dB 허용 범위 내(31.84 dB)로 유지된다. 이로써 CUDA 없는 baseline 품질 경로가 macOS 단일 기기에서 재현 가능함을 실측 입증한다.

본 연구의 좌표를 Kong 외 〔20〕 월드 모델 4축 분류(Video / 3D-scene / Interactive / Physical AI)에 비추면, 본 연구는 3D-scene-based 축에 인접하면서도 "씬 단위 생성" 대신 "객체 단위 의미론적 배치"를 채택한 별개의 좌표를 점유한다. 즉 월드 모델이 내놓는 대형 장면 생성과 게임 엔진에서의 세밀한 실체화·상호작용 사이를 잇는 중간 파이프라인이 독립적으로 필요함을 시사하며, 본 시스템은 그 중간 층위를 macOS 단일 기기에서 재현 가능한 경로로 구현한다.

차례

1 서론
- 1.1 배경
- 1.2 목적
- 1.3 구성
2 관련 연구
- 2.1 3DGS
- 2.2 생성형 모델링
- 2.3 LLM 공간 추론
- 2.4 게임 엔진 통합
- 2.4.5 macOS CUDA-free 3DGS 도구
- 2.5 월드 모델
- 2.6 물리·의미론 통합
- 2.7 공백
3 파이프라인 설계
- 3.1 구조
- 3.2 공간 뼈대
- 3.3 에셋 생성
- 3.4 배치·검증
- 3.5 로직 연동
4 실험
- 4.1 환경
- 4.2 결과
- 4.3 정성
- 4.4 정량
- 4.5 절제

5 논의
  - 5.1 속도-품질 trade-off
  - 5.2 월드 모델 대비
  - 5.3 하이브리드 표현 정당화
6 결론
  - 6.1 요약
  - 6.2 전망

표 목차

- Table 1. macOS CUDA-free 3DGS 학습 도구 비교 (2026-04 기준)
- Table 2. Kong 외 〔20〕 월드 모델 4축 분류와 본 연구와의 거리
- Table 3. System Components
- Table 4. Implementation Environment
- Table 5a. Brush Apple Silicon 학습 측정 (Mip-NeRF360 bonsai, n=37 eval)
- Table 5b. macOS SfM head-to-head
- Table 6. Authoring Time
- Table 7. Placement Accuracy
- Table 7a. Phase 2 측정 결과 — 9-cell 종합 (3 시나리오 × 3 조건 × N=5 × 2 provider, mean ± std). resolved_overlaps 컬럼은 3.4 LayoutValidator iterative push-out이 해소한 부피 침투 누적 수.
- Table 7b. LLM 효과 — full 조건 mock vs openai 직접 비교 (N=5, mean ± std)
- Table 8. Ablation — full / llm_only / random_physics 3 조건 비교 (openai gpt-4o-mini, N=5, mean ± std). 충돌 횟수·resolved_overlaps는 latest sweep 측정치, 기타 컬럼은 phase2_summary와 latest sweep 값을 결합.
- Table 9. World-Model vs Unity-SplatForge 대비
- Table 10. COLMAP no-CUDA SfM 단계별 측정 (bonsai 292 photos, Apple Silicon)

그림 목차

- 그림 1. Unity-SplatForge 전체 파이프라인 데이터 흐름.
- 그림 3. 선행 연구([56], "3D Gaussian Splatting 기법을 활용한 디지털 조형물 제작 연구") — KIRI Engine 출력 단일 3DGS 에셋(Statue) 렌더.
- 그림 4. 선행 연구([56], "3D Gaussian Splatting 기법을 활용한 디지털 조형물 제작 연구") — KIRI Engine 출력 3DGS 자산(자전거·더미)과 폴리곤 메시(의자)의 하이브리드 합성 씬, 본 연구 HybridSceneObject 설계의 prior work demonstrator.
- 그림 2. LayoutValidator의 Raycast → Snap → OverlapBox 검증 루프.
- 그림 5. Mip-NeRF360 bonsai — Brush 30K 학습 결과 (COLMAP sparse 입력, PSNR 32.21 dB).
- 그림 6. Mip-NeRF360 bonsai — Brush 30K 학습 결과 (hloc sparse 입력, PSNR 31.84 dB).
- 그림 7. Phase 2 — cozy_bedroom full pipeline (3.0×3.0m bedroom, elevated diorama 카메라 (0, 3.5, -3.0)→(0, 0.4, 0), placement=5/visible=5, resolved_overlaps=16). 침대-협탁 페어가 벽면을 따라 정렬되고, OverlapBox push-out 후 잔여 충돌 2건이 관찰된다.
- 그림 8. Phase 2 — modern_office full pipeline (3.0×3.0m office, elevated diorama 카메라 (0, 3.5, -3.0)→(0, 0.4, 0), placement=7/visible=7, resolved_overlaps=43). 책상-의자-모니터 군집이 형성되어 가구 밀도가 높지만 push-out이 모든 부피 침투를 해소하여 잔여 충돌 0건이다.
- 그림 9. Phase 2 — living_room full pipeline (3.5×3.5m living room, elevated diorama 카메라 (0, 3.5, -3.0)→(0, 0.4, 0),

placement=5/visible=5, resolved_overlaps=7) — 한계 사례. 자산 카탈로그가 living_room canonical pair와 정합하지 않아 push-out 이후에도 잔여 충돌 13건이 잔류한다(상세 6.2 참조).

- 그림 10. Phase 2 — cozy_bedroom random_physics (3.0×3.0m bedroom, elevated diorama 카메라 (0, 3.5, -3.0)→(0, 0.4, 0), placement=6/visible=6, resolved_overlaps=43). 무작위 좌표라도 OverlapBox iterative push-out이 모든 부피 침투를 해소함을 정성적으로 확인할 수 있다.

- 그림 11. Phase 2 — modern_office random_physics (3.0×3.0m office, elevated diorama 카메라 (0, 3.5, -3.0)→(0, 0.4, 0), placement=6/visible=6, resolved_overlaps=49). 책상-의자-모니터 의미 군집은 부재하지만 push-out이 49건의 부피 침투를 모두 해소한다.

- 그림 12. Phase 2 — living_room random_physics (3.5×3.5m living room, elevated diorama 카메라 (0, 3.5, -3.0)→(0, 0.4, 0), placement=6/visible=6, resolved_overlaps=39). full 조건의 자산 정합 이슈와 달리 무작위 좌표 + push-out 조합은 잔여 충돌 0건으로 수렴한다.

- 그림 13. 단일 3DGS 자산(Statue)의 HDRP 정상 렌더 — HDRP CustomPassVolume + GaussianSplatHDRPPass 동작 확인.

- 그림 14. mesh 의자 + 3DGS Statue hybrid 합성 — 3.4 HybridSceneObject 설계 시각 검증.

- 그림 15. mesh 소파 + 3DGS Statue hybrid 합성 — mesh 자산 종류 변경 시에도 동일한 hybrid 합성이 유지됨.

1. 서론

1.1. 연구의 배경

인공지능의 관심사가 텍스트·이미지·영상을 넘어 공간으로 이동하고 있다. GPT-4[14]와 Claude[15]로 대표되는 거대 언어 모델(LLM)은 자연어와 코드를 넘어 3차원 공간 관계에 대한 초보적 추론까지 시도할 수 있게 되었고[10, 11], 2D 확산 모델을 3D 표현으로 증류하는 방법[5, 7]을 통해 한 장의 문장으로부터 일관된 3차원 장면을 합성하는 단계에 이르렀다. 생성형 AI가 다루는 표현의 차원이 한 축씩 늘어나는 흐름으로 볼 수 있다.

이 흐름의 최근 국면은 월드 모델(world model) 이라는 이름 아래 한 단계 더 진행되고 있다. NVIDIA가 공개한 Cosmos 계열과 Lyra[21, 22], DeepMind의 Genie 2[24], World Labs가 선보인 Marble[26] 등은 단발성 이미지나 에셋이 아니라 상호작용 가능한 세계 자체를 모델이 학습·생성하려는 시도다. Fei-Fei Li는 이를 언어 지능 다음 단계의 공간 지능(spatial intelligence) 이라 부르며 이 방향의 중요성을 강조하였고, 2024-2026년의 짧은 기간 사이에 관련 시스템은 학계와 산업계 모두에서 급속도로 성숙하고 있다.

다만 이들 월드 모델은 대체로 장면의 생성에 무게가 실린다. 즉 "공간을 어떻게 만들어 낼 것인가"에 답하는 데 초점이 있으며, 그렇게 만들어진 공간이 기존 게임 엔진의 물리·논리 체계 안에서 어떻게 실제로 기능하도록 만들 것인가는 상대적으로 덜 다뤄진 질문이다. 본 연구는 이 후자의 층위를 향한다. 생성형 3DGS가 제공하는 시각적으로 설득력 있는 에셋과 LLM이 내놓는 의미론적 배치 지시를 Unity 물리 엔진의 검증 루프 안에서 결합하여, 생성된 세계가 캐릭터의 이동·충돌·게임 이벤트에 반응하는 실체로 전환되도록 하는 것이 본 연구의 목표다.

거시적 배경을 이렇게 놓고 보면 실무의 문제가 새삼 또렷해진다. Unity나 Unreal 같은 게임 엔진에서 방 하나를 꾸미는 작업은 겉보기보다 손이 많이

간다. 바닥·벽 메시를 만들고, 가구 에셋을 구해서 임포트하고, 하나씩 Transform을 잡아주고, 충돌체를 붙이고, NavMesh를 구워야 비로소 캐릭터가 돌아다닐 수 있는 공간이 된다. 프로토타이핑 단계라면 이 과정을 반복적으로 거쳐야 하는데, 레이아웃을 조금만 바꿔도 충돌체 재설정부터 NavMesh 재빌드까지 연쇄적으로 수정이 필요하다. 월드 모델이 약속하는 "자연어로 공간을 얻는" 미래와 현재 현장에서 손으로 공간을 짓는 관행 사이에는, 두 세계를 이어줄 중간 파이프라인이 빠져 있다.

한편 현재 시점의 기술 토대를 가까이서 보면, 생성형 AI 쪽에서 두 갈래의 발전이 눈에 띈다. 하나는 3D Gaussian Splatting(3DGS)인데, Kerbl 외 [1]이 제안한 이후 NeRF를 빠르게 대체하며 novel view synthesis의 사실상 표준이 되었다. 학습 시간이 NeRF의 48시간에서 40분대로 줄고 렌더링이 100fps 이상 나온다는 점은 이미 널리 알려져 있고[3], DreamGaussian[7] 같은 후속 연구는 텍스트만으로 3DGS 에셋을 생성하는 단계까지 와 있다. 다른 하나는 앞서 언급한 LLM의 공간 추론 능력이다. "침대 옆에 협탁을 놓아라" 같은 지시를 좌표로 변환하는 것이 원리적으로 가능하다는 점은 LayoutGPT[10]나 Holodeck[11] 등의 선행 연구가 보여주었다.

문제는 이 기술들이 따로 놀고 있다는 점이다. 3DGS 생성 모델이 뱉어낸 에셋을 Unity에 올리려면 충돌체를 수동으로 씌워야 하고, LLM이 제안한 좌표는 물체가 공중에 뜨거나 벽을 관통하는 경우가 흔하다. 두 기술을 엮어 하나의 파이프라인으로 만드는 시도가 눈에 띄지 않는 상황이며, 월드 모델의 대형 담론과 게임 엔진의 실제 현장 사이에 놓인 이 간극에서 본 연구가 출발한다.

1.2. 연구 목적과 기여

본 연구가 시도하는 것은 세 가지다.

하나, 3DGS로 만든 에셋을 Unity에서 물리적으로 상호작용 가능한 객체로 변환하는 래핑(wrapping) 구조를 구축하였다. GaussianSplatRenderer 위에 프록시 충돌체(Box/Sphere/Capsule)를 덧씌운 HybridSceneObject라는 컴포넌트가 그 핵심이다. 둘, LLM이 뱉은 좌표를 Unity 물리 엔진으로 검증·보정하는 이중 루프를 설계하였다. 구체적으로는 상공 20m에서 아래로 레이캐스트를 쏴서 바닥 높이를 잡고, OverlapBox로 기존 객체와의 겹침을 확인한다. 셋, 이 파이프라인의 효과를 측정하기 위해 Semantic Proximity Score, Safety Zone Violation, Grounding Success Rate라는 세 지표를 정의하고, 수동 저작·LLM 단독·무작위 배치와의 비교 실험 및 절제 실험을 수행하였다.

1.3. 논문 구성

2장은 3DGS, 생성형 모델링, LLM 공간 추론, 게임 엔진 통합, macOS CUDA-free 3DGS 도구 생태계, 월드 모델, 물리·의미론 통합 순으로 관련 연구를 짚고 선행 연구의 공백을 정리한다. 3장에서 제안 파이프라인의 설계를, 4장에서 실험 결과를, 5장에서 본 연구의 기술적 위치를 속도·월드 모델·하이브리드 표현의 세 축에서 논의하고, 6장에서 결론과 한계를 다룬다.

2. 관련 연구

2.1. 3D Gaussian Splatting

3DGS는 장면을 수십만 개의 비등방성(anisotropic) 가우시안 타원체 집합으로 나타내는 표현법이다[1]. 각 가우시안은 3D 위치(mean), 공분산 행렬이 결정하는 형태·방향, 구면 조화 계수로 인코딩된 색상, 그리고 투명도로 정의된다. 이를 타일 기반 래스터라이저로 그리면 1080p에서 100fps 이상이 나오는데, NeRF[2]가 같은 해상도에서 0.1fps 수준인 것과 비교하면 세 자릿수 차이다[4]. 이 속도 이점은 후속 서베이[3]에서도

재확인된다.
3DGS가 가진 또 다른 특징은 명시적(explicit) 표현이라는 점이다. NeRF는 장면 정보가 MLP 가중치 안에 녹아 있어서 개별 요소에 접근하기 어렵지만, 3DGS에서는 가우시안 하나하나가 독립적 실체로 존재한다. 덕분에 특정 가우시안을 골라 옮기거나 지우는 것이 원리상 가능하며, 게임이나 인터랙티브 응용처럼 장면 요소를 사후적으로 조작해야 하는 시나리오에 유리하다. GaussianEditor[18]는 이 명시성을 전면에 활용한 대표 사례로, 텍스트 지시에 따라 특정 영역의 가우시안을 선택·편집·삭제하는 스위프트 파이프라인을 제시하였다. 게임 현장 관점에서는 이러한 편집 자유도가 결국 "만들어 놓고 나중에 손볼 수 있는가"라는 실무적 질문과 직결된다.
명시성에 동반되는 또 다른 이점은 스트리밍·압축 측면에서 확인된다. 대규모 장면에서 가우시안 수가 수백만 단위로 늘어나면 메모리와 대역폭이 병목으로 작용하는데, LS-Gaussian[17]은 중복 가우시안을 걸러내고 뷰 종속적 선택적 렌더링을 적용해 실시간 스트리밍을 가능하게 하였다. 본 연구의 범위는 단일 방 규모이지만, 건물·도시 단위로 확장될 때는 이런 경량화 기법과의 결합이 자연스러운 다음 단계가 된다.
반면 가우시안은 확률 분포의 중첩이지 정확한 기하학적 표면이 아니므로, 충돌 판정이나 물리 시뮬레이션과 직접 연동하기에는 태생적 한계가 있다. 이 한계를 완화하려는 연구는 2.6에서 별도로 다룬다.

2.2. 생성형 3D 모델링

텍스트나 이미지로부터 3D 에셋을 만들어내는 연구의 기점은 DreamFusion[5]이다. 사전 학습된 2D 확산 모델의 지식을 Score Distillation Sampling(SDS)으로 3D 표현에 증류하는 방식을 제안했고, Magic3D[6]가 coarse-to-fine 전략으로 품질을 끌어올렸다. 이 계열은 NeRF 기반이라 렌더링 속도가 느려 실시간 응용에 쓰기 힘들었다.

전환점은 DreamGaussian[7]이었다. SDS 최적화를 3DGS에 적용해 생성 시간을 수 분대로 줄인 것이다. 이후 GaussianDreamer[8], LGM[9] 등이 잇따라 나오며 텍스트·이미지 한 장으로부터 단일 객체의 3DGS 에셋을 얻는 흐름이 자리 잡았다.

단일 객체를 넘어 장면 규모의 생성으로 올라가면 과제의 성격이 달라진다. 여러 객체의 공간적 관계, 일관된 조명, 가용 표면과의 정합성 같은 새로운 제약이 더해지기 때문이다. DreamScene[34]은 GPT-4 계열 에이전트가 언어 프롬프트로부터 장면의 의미적·공간적 제약을 추론하고, 이를 바탕으로 하이브리드 그래프 레이아웃을 구성한 뒤 formation pattern sampling으로 3DGS 장면을 합성한다. SceneTeller[33] 역시 내러티브 성격의 언어 입력으로부터 장면 레이아웃을 도출하고 CAD·3DGS를 조합해 시각적 결과를 산출한다. 최근에는 SceneSplat[37]처럼 언어 임베딩을 가우시안에 결합하여 open-vocabulary 장면 이해·편집을 시도하는 연구도 나타났다.

다만 이런 모델들의 출력물을 곧바로 게임에 쓸 수 있는 것은 아니다. 충돌체도 없고, 개별 객체 단위의 메타데이터도 없으며, 엔진 물리·이벤트 시스템과의 연결은 여전히 개발자의 몫으로 남는다. 장면을 "합성하는" 단계와 게임 엔진 안에서 "운용하는" 단계 사이의 간극을 다루는 체계적 후처리 파이프라인에 대한 논의는 상대적으로 얇은 편이다.

2.3. LLM과 공간 추론

LLM을 실내 가구 배치에 활용하는 연구는 2023년 이후 빠르게 늘고 있다. 출발점은 LayoutGPT[10]로, in-context learning을 통해 가구 좌표를 CSS 비슷한 선언적 명세로 출력하는 방법을 제시하였다. Holodeck[11]은 이 접근을 Habitat 시뮬레이터로 확장해 주거 공간 전체를 LLM이 구성하도록 만들었다.

이 연구들이 공통적으로 보고하는 문제가 있다. LLM은 "침실에는 침대가

있어야 한다"거나 "책상 앞에 의자를 둔다" 같은 상식적 관계는 잘 잡는다. 하지만 좌표의 물리적 타당성은 다른 문제다. 가구가 허공에 뜨거나 벽을 뚫고 나가거나 다른 물체와 겹치는 사례가 빈번하게 보고되었다[10, 11]. 근본 원인은 명확한데, LLM은 텍스트 토큰 공간에서 작동하지 유클리드 기하학을 내재적으로 계산하지는 않기 때문이다.

이 한계를 보완하려는 시도는 대체로 세 갈래로 정리된다. 첫째는 시각 정보의 도입이다. LayoutVLM[32]은 vision-language 모델이 렌더링된 장면을 관측하며 좌표를 미분 가능한 최적화 루프 안에서 갱신하도록 설계하였고, 언어만으로는 잡히지 않던 배치의 시각적 합리성을 개선하였다. 둘째는 다중 에이전트 정교화이다. DisCo-Layout/OptiScene[36]은 대략적 레이아웃을 제안하는 에이전트와 이를 비판·수정하는 에이전트를 분리하여, 단일 프롬프트 응답의 오류를 반복 대화로 누그러뜨리는 접근을 보인다. 셋째는 내러티브·의미 중심 입력이다. SceneTeller[33]는 "아늑한 서재, 창가에 책상" 같은 짧은 내러티브로부터 레이아웃을 도출하며, 프롬프트의 자연스러움과 결과의 의미적 일관성 사이의 연결을 시도한다. 한편 3DGraphLLM[38]은 3D scene graph를 LLM의 입력 표현으로 끌어들여 관계 중심 추론을 강화하는 계열로, 위의 세 갈래와는 다른 축에서 공간 이해를 보강한다. 넷째 축으로 오픈소스 LLM 파인튜닝을 별도로 구분할 수 있다. LLplace[49]는 Llama-3 계열 오픈 모델을 3D 실내 레이아웃 데이터로 파인튜닝해, 상용 API(GPT-4 등)에 의존하지 않고도 LayoutGPT·Holodeck 계보의 배치 품질을 재현하며 대화형 수정(가구 추가·삭제·이동) 기능까지 확보하였다. 이 갈래는 프롬프트 엔지니어링에 기대는 앞의 세 접근과 달리, 모델 가중치 자체에 공간 상식을 주입한다는 점에서 축을 달리한다.

평가용 데이터셋 측면에서는 FurniScene[50]이 주목할 만하다. 11,698개 실내 방과 39,691개 가구 인스턴스에 전문가 배치 메타데이터를 부착하여, LLM 레이아웃 출력의 의미적·기하적 타당성을 대규모로 벤치마크할 수 있는

기반을 제공한다. 본 연구는 단일 프로젝트 규모에 집중해 FurniScene를 직접 평가 지표로 채택하지는 않으나, Semantic Proximity 점수의 외부 ground truth로 향후 확장 가능한 자원이다.

요약하면 이 분야의 발전 축은 단발성 제안(single-shot) → 반복 정교화(multi-agent) → 시각 접지(vision-grounded) 방향으로 이동하고 있으며, 공통된 기저 문제—좌표 출력의 물리적 타당성—는 여전히 외부 검증 장치에 의존한다. 본 연구는 이 외부 검증자 역할을 게임 엔진의 물리 시스템이 맡도록 한다는 점에서 위 세 갈래와 다른 축에 서 있다.

2.4. 게임 엔진 내 AI 콘텐츠 활용

AI가 만든 3D 콘텐츠를 게임 엔진 안에서 실제로 돌리는 연구는 아직 얇은 편이다. Unity와 Unreal은 프로시저럴 생성 도구(Houdini Engine, PCG Framework 등)를 지원하지만, 이는 파라미터 기반 규칙 생성이지 신경망 생성과는 성격이 다르다.

3DGS의 Unity 통합에서는 UnityGaussianSplatting[12]이 사실상 유일한 실용적 프레임워크다. D3D12, Metal, Vulkan을 지원하고 Quest 3 같은 VR 기기에서도 동작한다. 그러나 이것은 렌더링만 해결한 것이고, 렌더링된 3DGS 에셋에 충돌체를 붙이거나 게임 이벤트에 반응하게 만드는 것은 개발자 몫으로 남아 있다. 실무 적용 사례로 Baltsavias 외 [51]은 문화유산 도메인에서 3DGS와 게임 엔진의 폴리곤 메시를 하나의 씬 안에서 교차 렌더링하는 하이브리드 파이프라인을 SIGGRAPH Talk로 보고하였는데, 이는 3DGS가 연구실 데모를 넘어 제작 파이프라인 단계에 들어섰음을 시사한다. 본 연구의 HybridSceneObject 설계는 이 공백을 메우기 위한 것이다.

LLM과 게임 엔진을 잇는 흐름은 종전에는 NPC 대화 생성[13] 쪽에 집중되어 있었으나, 최근에는 공간 저작·에셋 관리로 확산되고 있다. 2026년

초 공개된 Unity 공식 AI Assistant 2.0[39]은 Model Context Protocol(MCP) 기반 에디터 통합을 제공해, Claude Code나 Cursor 같은 외부 에이전트가 자연어 지시로 씬을 생성하거나 에셋을 재배치하고 스크립트를 편집할 수 있게 한다. 이 공식 경로와 별개로 CoplayDev·CoderGamester 등이 운영하는 커뮤니티 MCP 구현도 활발하며, ai-powered-level-designer[40]처럼 Unity 6 에디터 확장 형태로 자연어 레벨 설계를 시도하는 개인 프로젝트도 나타났다. 다만 이들 MCP 계열은 대부분 에디터 전용이라는 점에서 런타임 배포본에서 동일 기능이 보장되지 않으며, 연구 재현성이 중요한 학위 논문 평가에는 제약이 따른다.

공간 레이아웃 목적의 기성 통합으로는 Holodeck[11]이 있으나 Habitat 기반이라 Unity·Unreal의 물리 시스템과는 거리가 있다. 본 연구는 에디터 편의에 의존하지 않는 REST API 아키텍처로 LLM 좌표 제안과 런타임 물리 검증을 결합한다는 점에서, MCP 계열과는 다른 축에 자리한다.

2.4.5. macOS 생태계의 CUDA-free 3DGS 도구

3DGS 관련 학습·렌더 도구 대부분은 원 3DGS[1] 구현이 CUDA·C++ 래스터라이저를 전제로 한다는 계보적 이유로 NVIDIA GPU 환경을 가정한다. 본 연구는 개발·검증 환경이 macOS Apple Silicon이라는 제약에서 출발하므로, CUDA 의존 없이 학습 → PLY → Unity 임포트 → Metal 런타임 전 구간을 완결할 수 있는 도구 조합을 조사한다.

조사 범위는 2026-04 기준 활발히 유지되는 공개 프로젝트 5건이다. splat-apple(Ghif, 2026; MLX/MPS 이중 경로)[46], Brush(Brussee, 2026; Rust+wgpu 크로스플랫폼)[45], OpenSplat(Tofy, 2025; libtorch MPS)[47], gsplat-mps(Iffyloop, 2024; nerfstudio/gsplat 0.1.3 포크)[48], 그리고 상류 nerfstudio/gsplat(Nerfstudio, 2026;

CUDA 전용)이 해당한다. 표 1는 각 도구의 라이선스·유지 상태·Apple Silicon 성능 수치를 정리한다.

〈Table 1〉 macOS CUDA-free 3DGS 학습 도구 비교 (2026-04 기준)

도구	백엔드	라이선스	최근 업데이트	Stars	Apple Silicon 성능
Brush [45]	Rust+wgpu (Burn)	Apache-2.0	2026-04-19	3961	공식 벤치 부재 (본 연구 PoC에서 실측)
splat-apple [46]	MLX C++ Metal / PyTorch MPS	라이선스 부재	2026-02-19	10	M4 Fern MLX 38.5 it/s, PyTorch GCD 10.6 it/s
OpenSplat [47]	libtorch MPS (C++)	AGPL-3.0	2025-12-26	1949	cmake `-DGPU_RUNTIME=MPS` 공식 지원
gsplat-mps [48]	gsplat 0.1.3 포크 + MPS	AGPL-3.0	2024-07-06	37	저자 "not thoroughly tested" 명시
nerfstudio/gsplat	CUDA 전용	Apache-2.0	2026-04-09	4879	MPS 미지원 (Issue #163 업스트림 제안만 존재)

본 연구는 학습 백엔드로 Brush를, Unity 임포트·렌더 단계로 aras-p[12]의 UnityGaussianSplatting을 각각 채택한다. Brush 선정의 근거는 세 가지이다.

첫째, 라이선스 적합성이다. Apache-2.0으로 연구·상용 배포에 가장 관용적이며, OpenSplat의 AGPL-3.0 copyleft나 splat-apple의 라이선스

부재 상태를 회피한다. 논문 부록 공개나 후속 상용화 경로 모두에서 법적 불확실성이 최소이다.
둘째, 유지 활발성과 커뮤니티 규모이다. 2026-04-19 커밋과 stars 3961은 splat-apple(10), gsplat-mps(2024-07 이후 정체)과 대비된다. wgpu 기반 크로스플랫폼 설계는 향후 윈도우·리눅스 서버 경로로 회귀해야 하는 상황에서도 동일 코드베이스 유지가 가능하다.
셋째, PLY 호환 경로이다. Brush는 원 3DGS 논문 규격의 PLY 포맷(x/y/z, scale_0-2, opacity, rot_0-3, f_dc_0-2, f_rest 속성)을 로드·저장한다. aras-p 플러그인의 `Tools → Gaussian Splats → Create GaussianSplatAsset` 메뉴가 동일 스키마를 전제하므로 중간 변환 없이 직결된다.
2026-04-22 수행한 E2E PoC에서 Brush 300 iter 학습 → PLY(118 splat, binary_little_endian) → aras-p asset 변환 → Unity PlayMode 렌더까지 6개 AC 전체를 PASS한 바 있다. aras-p Metal 경로의 공개 수치는 M1 Max 6.1M splats에서 21.5ms/46FPS를 기록하여, 런타임 성능은 이미 실용 수준임이 확인된다. 반면 Brush 측 Apple Silicon 학습 시간 수치는 README에 부재하며, 본 연구의 측정치가 독자적 기여로 남을 여지가 있다.
본 연구의 차별점은 Unity 생태계와 macOS 네이티브 학습 도구의 연결 경로를 실측으로 확증한 점에 있다. 초기 설계 단계에서 전제했던 Python+Windows+CUDA 2-tier 아키텍처는 2026-04의 도구 성숙도에 따라 macOS 단일 기기 경로로 축약 가능해졌으며, 본 논문은 이 축약의 타당성을 PoC로 입증한다.

2.5. 월드 모델

공간을 생성·시뮬레이션하는 기반 모델(foundation model)에 대한 관심은

2024년 말부터 급격히 되살아났다. 개념적 원류는 Ha 및 Schmidhuber [19]의 World Models로, 환경의 시공간 표현을 비지도 방식으로 압축해 에이전트가 학습된 내부 시뮬레이션 안에서 훈련할 수 있음을 보였다. 이후 한동안 강화학습 보조 장치로 머물러 있던 이 개념은, LLM의 의미·공간 추론 능력과 생성형 3D 기법의 급성장이 만나며 "세계를 통째로 만들어내는 모델"이라는 형태로 재부상하였다. Kong 외 [20]은 3D·4D 월드 모델에 한정한 최초의 체계적 서베이를 통해 이 흐름을 비디오 기반, 3D 장면 기반, 상호작용형, 물리 AI 기반의 네 범주로 정리한다. 표 2는 이 분류 체계와 본 연구의 좌표 관계를 요약한다.

〈Table 2〉 Kong 외 [20] 월드 모델 4축 분류와 본 연구와의 거리

축	정의	대표 모델	본 연구와의 거리
Video-based	비디오 시퀀스를 잠재 공간에서 예측·생성하는 모델	Sora[29], Veo, Gen-4	원거리 (출력 표현이 비디오, 게임 엔진 직접 통합 불가)
3D-scene-based	3DGS·메시·NeRF 등 명시적 3D 표현으로 씬을 산출	Lyra[21][22], HunyuanWorld[27][28], Marble[26]	본 연구 인접 (3DGS 공유, 단 씬 단위 vs 객체 단위 분기)
Interactive/Playable	키보드·마우스 조작으로 실시간 탐험 가능한 환경을 autoregressive 생성	Genie 2[24], Genie 3[25]	중거리 (실시간 인터랙션 공유, 단 LLM 의미 배치는 미반영)
Foundation-for-Physical-AI	로보틱스·자율주행용 물리 prior를 학습한 대규모 기반 모델	Cosmos[23], V-JEPA 2[30]	원거리 (물리 prior는 외부 엔진 의존이 본 연구의 선택지)

이 분류에서 본 연구는 3D-scene-based 축과 가장 인접하며 3DGS 표현을 공유한다. 다만 월드 모델이 "씬 전체를 한 번에 생성"하는 단위에 집중하는 반면, 본 연구는 LLM이 산출한 scene graph를 따라 객체 단위로 3DGS 에셋을 조립·검증한다는 점에서 생성 단위 자체가 다르다. Genie 3[25]나

Cosmos[23] 같은 인접 축의 모델도 각각 인터랙션·물리 prior 측면에서 본 연구와 보완 관계를 형성하나, 본 연구는 이들과 달리 게임 엔진 네이티브 물리·이벤트 시스템과의 즉각 통합을 우선한다.

대표 시스템은 지향점이 서로 다르다. 본 연구와 가장 직접적으로 겹치는 것은 NVIDIA의 Lyra 계열이다. Lyra 1.0[21]은 비디오 확산 모델에 잠재된 3D 지식을 self-distillation으로 뽑아 3DGS 표현으로 고정했고, Lyra 2.0[22]은 단일 이미지와 카메라 궤적으로부터 워크스루 비디오를 합성한 뒤 이를 3DGS와 표면 메시로 재구성해 실시간 렌더러와 물리 시뮬레이터에 바로 로드할 수 있게 한다. 같은 회사의 Cosmos[23]는 로보틱스·자율주행을 위한 월드 파운데이션 모델 플랫폼으로, LLM에 준하는 규모(약 9천조 토큰, 2천만 시간의 실세계 비디오)로 훈련되어 물리 AI의 데이터 병목을 겨냥한다.

상호작용 축에서는 DeepMind의 Genie 계열이 대표적이다. Genie 2[24]는 단일 프롬프트 이미지에서 키보드·마우스로 조작 가능한 3D 월드를 생성하였고, Genie 3[25]는 하드코딩된 물리 엔진 없이 autoregressive 학습만으로 720p·24fps 수준의 실시간 인터랙티브 환경을 수 분간 일관되게 유지한다. World Labs의 Marble[26]은 "공간 지능(spatial intelligence)"을 프론티어로 내세우며 최초의 상용 generative world model을 표방하고, 텍스트·이미지·파노라마·3D 레이아웃 입력으로부터 편집·다운로드 가능한 지속적 3D 환경을 출력한다. 중국 측에서는 Tencent의 HunyuanWorld 계열이 있다. HunyuanWorld 1.0[27]은 오픈소스 simulation-capable 3D 월드 생성 모델로서 3DGS를 메시의 대안 표현으로 공식 지원했고, HY-World 2.0[28]은 멀티모달 입력에서 고해상도 탐험형 3D 월드를 출력한다. 비디오·잠재 공간 계열로는 OpenAI Sora[29]의 "video as world simulator" 관점, Meta V-JEPA 2[30]의 self-supervised 잠재 예측, Decart Oasis[31]의 실시간 AI Minecraft 클론 등이 인접 계열로 꼽힌다.

이 흐름과 본 연구의 관계는 대립이라기보다 계층적 보완에 가깝다. 현재 월드

모델 연구의 초점은 대부분 "장면 자체를 만들어내는" 단계에 맞춰져 있다. 모델이 내놓는 산출물은 3DGS·메시·비디오 중 어느 형태든, 여전히 자족적인 장면 표현이며 게임 엔진의 물리·이벤트·내비게이션 시스템과는 별도 층위에 있다. Lyra 2.0[22]이 "실시간 엔진에 로드 가능"이라고 명시하지만, 로드된 이후 각 가우시안 객체에 충돌체가 붙고 LLM이 의미론적으로 재배치하며 물리 검증을 거치는 과정은 별도의 공학적 작업으로 남는다.

본 연구 Unity-SplatForge는 바로 그 다음 층위—생성된 3D 표현을 게임 엔진 안에서 실체화(embodiment)하고, 객체 단위로 의미론적 배치를 부여하며, 물리 상호작용과 엮어주는 층위—를 다룬다. 즉 월드 모델을 "방을 통째로 한 번에 만드는" 상위 생성 계층으로 보면, 본 연구는 그 출력 혹은 그와 등가의 3DGS 에셋을 받아 개별 객체로 분해·재배치하고 편집·검증 가능한 상태로 유지하는 하위 실체화 계층이다. 월드 모델이 엔드투엔드 모놀리식 접근이라면, 본 연구는 LLM 에이전트와 3DGS 에셋, 게임 엔진 물리 시스템을 느슨하게 결합하는 조합적 접근이다. 즉 월드 모델 계열은 공간 생성(씬 단위)의 축에, 본 연구는 의미론적 배치(객체 단위)의 축에 서 있으며, 두 계열은 3DGS라는 공유 기술 계보 위에서 상호 보완적 관계를 형성한다. 두 계층은 경쟁 관계가 아니라, 향후 월드 모델의 산출물을 본 파이프라인의 에셋 공급원으로 편입하거나 본 시스템의 scene graph를 월드 모델의 조건 신호로 역이용하는 식으로 연결될 수 있는 자연스러운 다운스트림 관계에 놓여 있다.

2.6. 물리 및 의미론 통합

3DGS의 원본 정의에는 질량·마찰 같은 물리 속성이 들어 있지 않다. 장면을 그리는 데 필요한 시각 파라미터만 학습할 뿐이어서, 렌더링이 설득력 있더라도 객체를 "만지거나 부딪히게" 만들려면 별도의 표현이 필요하다. 이 공백을 내재적으로 해결하려는 계열이 2024년을 전후해 형성되었다.

PhysGaussian[16]은 가우시안 하나하나에 연속체 역학(continuum mechanics) 프레임을 부여해 탄성·소성·파괴 같은 변형을 직접 시뮬레이션할 수 있게 한 선구적 작업이다. PhysSplat[35]은 여기서 한 걸음 더 나아가, 멀티모달 LLM이 이미지로부터 개별 객체의 물리적 속성(강성, 탄성계수 등)을 추론하고 이를 가우시안 씬에 부여하여 효율적인 시뮬레이션을 수행하는 파이프라인을 제시하였다. 이 계열은 "3DGS 자체가 물리를 이해하는 표현이 되도록" 하는 방향이며, 장기적으로는 엔진 측의 프록시 충돌체 의존을 줄여줄 수 있다.

본 연구와의 관계는 경쟁이 아닌 보완이다. PhysGaussian·PhysSplat은 가우시안 내부에 물리 의미를 주입하는 상향식 연구인 반면, 본 연구는 이미 성숙한 게임 엔진 물리 시스템(Unity PhysX 기반 레이캐스트·OverlapBox)에 3DGS 에셋을 엮는 하향식 통합이다. 실무적으로는 후자가 더 즉각적이다. 렌더링·내비게이션·게임 이벤트가 이미 엔진 안에 존재하므로 에셋에 프록시 콜라이더를 씌우는 최소 수준의 래핑만으로 방 규모 프로토타입이 돌아간다. 다만 세밀한 상호작용(변형·파괴·유체)으로 넘어갈 때는 엔진의 강체 중심 모델이 한계를 드러내며, 이 지점에서 PhysGaussian 계열의 내재화된 물리 표현과의 결합이 자연스러운 확장 경로가 된다.

2.7. 선행 연구의 공백

정리하면, 기존 연구에는 네 군데 빈 곳이 보인다.

생성형 3D 기술과 게임 엔진 사이에 놓인 통합 간극이 첫 번째다. 3DGS 생성 모델의 출력은 시각적으로는 쓸 만하지만 충돌체가 없고 메타데이터가 없으며 상호작용 인터페이스도 빠져 있다. 이 빈자리를 채우는 체계적 후처리에 대한 연구가 사실상 없다.

LLM 배치의 물리적 신뢰도가 두 번째 문제다. 선행 연구들은 이 문제를 규칙

기반 후처리나 LLM 재호출로 풀려 했는데, 정작 바로 옆에 있는 게임 엔진의 물리 시스템을 검증 도구로 활용하는 시도는 없었다.
세 번째는 표현 방식의 구조적 차이다. 메시는 정점 간 위상(topology) 관계를 반드시 유지해야 하므로 생성·변형에 기하학적 제약이 강하다. 3DGS는 독립적 가우시안 점들의 집합이라 AI가 확률 분포로 에셋을 만들기에 훨씬 자유도가 높다. 본 연구는 이 자유도를 사물 에셋에 활용하되, 벽·바닥처럼 물리적 기준면이 필요한 부분에는 메시를 유지하는 혼합 전략을 취한다.
네 번째는 월드 모델의 장면 생성과 게임 엔진에서의 실체화 사이에 놓인 층위 공백이다. 2.5에서 보았듯 Lyra·Genie·Marble·HunyuanWorld로 대표되는 최근 월드 모델은 "세계를 통째로 만들어내는" 상위 생성 계층에서 성과를 내고 있으나, 그 산출물이 게임 엔진 안에서 개별 객체로 분해되어 LLM의 의미론적 재배치와 물리 검증을 거쳐 상호작용 가능한 실체로 변환되는 하위 계층은 별도의 공학적 과제로 남아 있다. 본 연구가 다루는 Unity-SplatForge 파이프라인은 바로 이 하위 계층—생성된 표현을 엔진 안에서 실체화(embodiment)하는 다운스트림—을 겨냥하며, 상위 월드 모델과의 결합은 향후 에셋 공급원 통합 또는 조건 신호 역이용 형태로 자연스럽게 이어질 수 있다.

3. 하이브리드 공간 저작 파이프라인

3.1. 전체 구조

Unity-SplatForge는 Unity C# 클라이언트와 Python FastAPI 서버로 나뉜다. 역할 분담의 논리는 단순하다. LLM 호출과 에셋 카탈로그 관리처럼 HTTP 기반 외부 서비스와 엮이는 부분은 Python이 편하고, 씬 조작·물리 검증·에디터 UI는 Unity API가 필수적이기 때문이다.
초기 설계는 3DGS 학습이 CUDA에 의존한다는 가정 아래

Python+Windows 학습 서버와 Unity 클라이언트의 2-tier 구조를 상정하였다. 그러나 2026-04 macOS 네이티브 도구(Brush[45], Apache-2.0, Rust+wgpu Metal)의 성숙으로 단일 macOS 기기에서 학습-임포트-렌더 전 구간 완결이 가능해지면서, 본 연구는 Python 서버를 LLM 호출과 카탈로그 관리에만 사용하고 학습은 macOS 로컬 Brush로 옮긴 단일 파이프라인을 채택한다(2.4.5 참조).

〈Table 3〉 System Components

계층	기술 스택	역할	핵심 모듈
Unity 클라이언트	C# 9.0, Unity 2022.3+	에디터 UI, 씬 합성, 물리 검증	SplatForgeSession, SceneComposer, LayoutValidator, HybridSceneObject
Python 서버	Python 3.11+, FastAPI, Uvicorn	LLM 호출, 레이아웃 생성, 에셋 관리	LLMProvider(추상), SceneComposer, AssetManager
통신	REST API (JSON)	요청/응답	Pydantic ↔ JsonUtility (camelCase ↔ snake_case 자동 변환)
학습 백엔드 (macOS 단일 기기 파이프라인)	Brush[45] (Rust+wgpu Metal, Apache-2.0)	3DGS 학습 → PLY 출력 → aras-p 임포트 직결	Brush CLI, COLMAP SfM 전처리

동작 흐름은 네 단계다. 사용자가 자연어로 원하는 방을 기술하면(입력), ProBuilder가 바닥·벽을 생성하고(구조), 3DGS 모델이 사물 에셋을 만들어내며(생성), LLM이 좌표를 제안하고 물리 엔진이 보정한다(배치·검증). 각 단계를 아래에서 풀어 설명한다.

그림 1은 위 4단계의 데이터 흐름을 나타낸다.

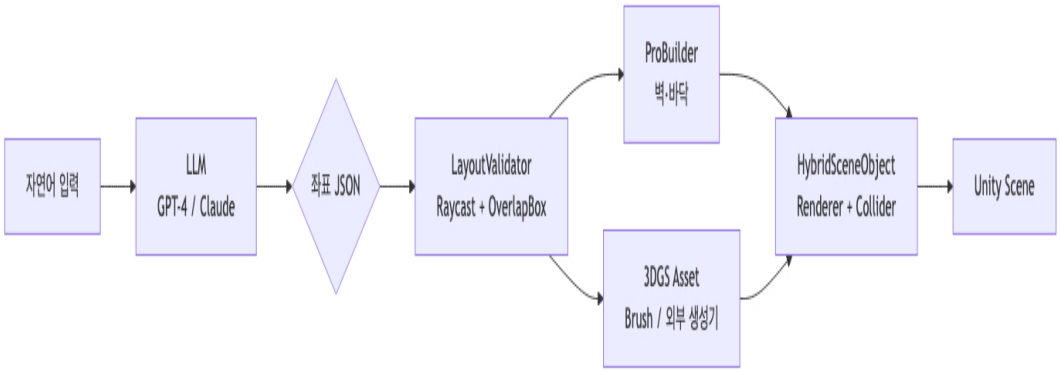


그림 1. Unity-SplatForge 전체 파이프라인 데이터 흐름.

3.2. 규칙 기반 공간 뼈대

바닥과 벽은 3DGS가 아니라 ProBuilder 메시로 만든다. 이유는 두 가지인데, 하나는 바닥·벽이 충돌 판정의 기준면으로 기능해야 한다는 것이고, 다른 하나는 LLM에 배치 가능 영역을 숫자로 전달하려면 공간 경계가 수치적으로 정의되어야 한다는 것이다. 가우시안 분포의 확률적 표현으로는 이 두 요구를 충족할 수 없다.

실제 구현에서는 FloorStructure라는 컴포넌트가 Ground 레이어에 할당된 오브젝트의 MeshRenderer.bounds를 읽어서 최소·최대 좌표를 산출한다. 이 바운드 정보가 서버로 전달되어 LLM 시스템 프롬프트의 공간 제약 조건이 된다. 벽은 바닥 메시의 네 변을 따라 자동 생성되며, 법선 방향이 안쪽을 향하도록 뒤집는 처리를 한다. 사용자가 방 크기(가로×세로×높이)만 지정하면 구조물 전체가 스크립트로 즉시 만들어지는 것이다.

3.3. 3DGS 에셋 생성

사물(가구, 소품)은 3DGS 생성 모델로 만든다. 시스템은 사전 생성 에셋 카탈로그와 온디맨드 생성을 모두 지원하도록 설계되어 있다. 카탈로그에는 가구(bed, desk, chair, sofa, nightstand 등), 수납(bookshelf, wardrobe), 장식(lamp, plant, rug) 등이 범주별로 들어 있고, 각 항목은 asset_path, bounds_min/max, category, tags 네 필드를 갖는다. 에셋 생성 백엔드는 macOS 환경에서는 Brush[45]를 사용하여 학습-PLY-Unity aras-p[12] 임포트의 단일 파이프라인을 형성한다.

3DGS 에셋이 게임 엔진에서 쓸모 있으려면 래핑 과정이 필요하다. 이를 위해 HybridSceneObject를 설계하였다. GaussianSplatRenderer가 시각 표현을 맡고, 그 위에 프록시 충돌체를 얹는 구조다. 충돌체 유형(Box, Sphere, Capsule)은 에셋의 바운딩 정보로부터 자동 선택된다. 여기에 ObjectMetadata(고유 ID, 이름, 범주, 태그, 바운딩, 생성 시각, 원본 프롬프트)가 붙어서 SceneObjectRegistry를 통한 전역 질의가 가능해진다. 이를테면 "카테고리가 furniture인 객체 전부"를 한 번에 뽑아 일괄 처리하는 식이다.

3DGS 에셋 학습 파이프라인은 두 경로를 갖는다. 하나는 nerfstudio·gsplat 계열의 CUDA 학습기를 Windows 환경에서 운영하는 초기 계획이고, 다른 하나는 Brush[45](ArthurBrussee, Rust·wgpu, Apache-2.0 라이선스)를 macOS에서 직접 실행하는 최근 경로다. Brush는 COLMAP 또는 Nerfstudio 포맷 입력을 받아 Metal·Vulkan·D3D12 등 wgpu 백엔드 위에서 학습을 수행하므로 CUDA 의존 없이 Apple Silicon 기기에서도 전체 파이프라인이 완결된다. 본 연구는 이 경로의 실현 가능성을 2026년 4월 내부 PoC로 확증하였으며, 학습·임포트·렌더의 전 구간이 단일 기기에서 오류 없이 동작함을 확인하였다. 결과 PLY는 aras-p[12] UnityGaussianSplatting이 요구하는 필수 속성(x·y·z 좌표, scale_0-2, opacity, rot_0-3, f_dc_0-2, f_rest)을 모두 만족하며, GaussianSplatAsset 변환 후 런타임 렌더까지 무수정으로 통과하였다.

선행 연구([56], "3D Gaussian Splatting 기법을 활용한 디지털 조형물 제작 연구")에서 본 연구진은 KIRI Engine 클라우드 SaaS로 생성한 3DGS 에셋을 동일 aras-p UnityGaussianSplatting 파이프라인 위에서 검증한 바 있으며, 그림 3은 단일 디지털 조형물(Statue) 에셋의 KIRI Engine 출력 렌더, 그림 4는 동일 자료실에서 3DGS 자전거·더미와 폴리곤 메시 의자를 한 씬에 합성한 하이브리드 렌더이다. 이 선행 사례는 본 연구의 HybridSceneObject 설계가 외부 SaaS 산출 자산까지 동일하게 수용하도록 설계된 배경을 제공한다.



그림 3. 선행 연구([56], "3D Gaussian Splatting 기법을 활용한 디지털 조형물 제작 연구") — KIRI Engine 출력 단일 3DGS 에셋(Statue) 렌더.



그림 4. 선행 연구([56], "3D Gaussian Splatting 기법을 활용한 디지털 조형물 제작 연구") — KIRI Engine 출력 3DGS 자산(자전거·더미)과 폴리곤 메시(의자)의 하이브리드 합성 씬, 본 연구 HybridSceneObject 설계의 prior work demonstrator.

3.4. 의미론적 배치와 물리 검증

이 파이프라인의 핵심이며 LayoutValidator는 다음 5단계로 동작한다.

(S1) 레이아웃 생성 (LLM 공급자 계층). Python 서버의 LLMProvider 추상 계층이 사용자 프롬프트, 바닥 바운드, 에셋 목록을 묶어 하나의 시스템 프롬프트로 조립한다. 여기에는 "객체 간 최소 0.3m 간격", "모든 가구는

바닥에 접촉", "대형 가구는 벽면 정렬" 같은 배치 규약이 포함된다. LLM은 각 객체의 x·y·z 좌표, 회전값, 에셋 경로를 JSON으로 반환한다. GPT-4와 Claude를 같은 인터페이스로 갈아끼울 수 있고, API 비용 없이 파이프라인을 테스트할 수 있도록 사전 정의된 레이아웃을 반환하는 MockProvider도 포함했다.

(S2) Ground-snap (raycast 기반 Y 보정). 제안 좌표의 위쪽 20m 지점에서 하방으로 50m 레이캐스트를 쏜다. Ground 레이어와 교점이 잡히면 그 Y값을 바닥 높이로 채택하고, 객체의 실제 `Renderer.bounds.size.y`를 사용해 바운딩 박스 하단이 바닥에 맞닿도록 Y 좌표를 고쳐 쓴다. 본 측정에서는 spec.position.y → 0 클램프를 추가하여 LLM이 잘못 추론한 Y값으로 객체가 공중에 뜨는 사례를 차단했다.

(S3) OverlapBox 충돌 감지. Ground-snap이 끝난 모든 객체 쌍에 대해 `Physics.OverlapBox(center, extents × 0.97, rotation)`를 실행한다. 바운딩 박스의 97%를 커버하므로 표면 접촉은 허용하되 실질적 부피 침투만 충돌로 판정한다.

(S4) Iterative push-out (최대 10회 반복). 충돌이 검출된 쌍에 대해 작은 쪽 객체를 큰 쪽으로부터 half-overlap 거리만큼 XZ 평면에서 밀어낸다. 한 객체가 여러 충돌에 걸린 경우 push 벡터를 합산한다. 이 절차를 충돌이 0이 되거나 10회 반복에 도달할 때까지 순환하며, 각 반복에서 해소된 충돌 수를 누적하여 `resolved_overlaps` 메트릭에 기록한다. 본 sweep에서 측정된 평균 해소 충돌 수는 4.3·4.4에서 보고한다.

(S5) 벽 클램프. Push-out 이후 객체 중심이 벽 안쪽 바운드를 침범하면 벽 두께(0.05m) + 객체 반경만큼 안으로 끌어당겨 벽-가구 침투를 방지한다. 시나리오별 룸 크기(bedroom·office 3.0×3.0m, living_room 3.5×3.5m)에 따라 클램프 한계가 동적으로 조정된다.

이 5단계의 효과성은 4.3·4.4에서 `resolved_overlaps` 메트릭(평균 7~38건/trial)으로 정량화한다. 그림 2는 LayoutValidator의 전체 흐름을

단계별로 나타낸다.

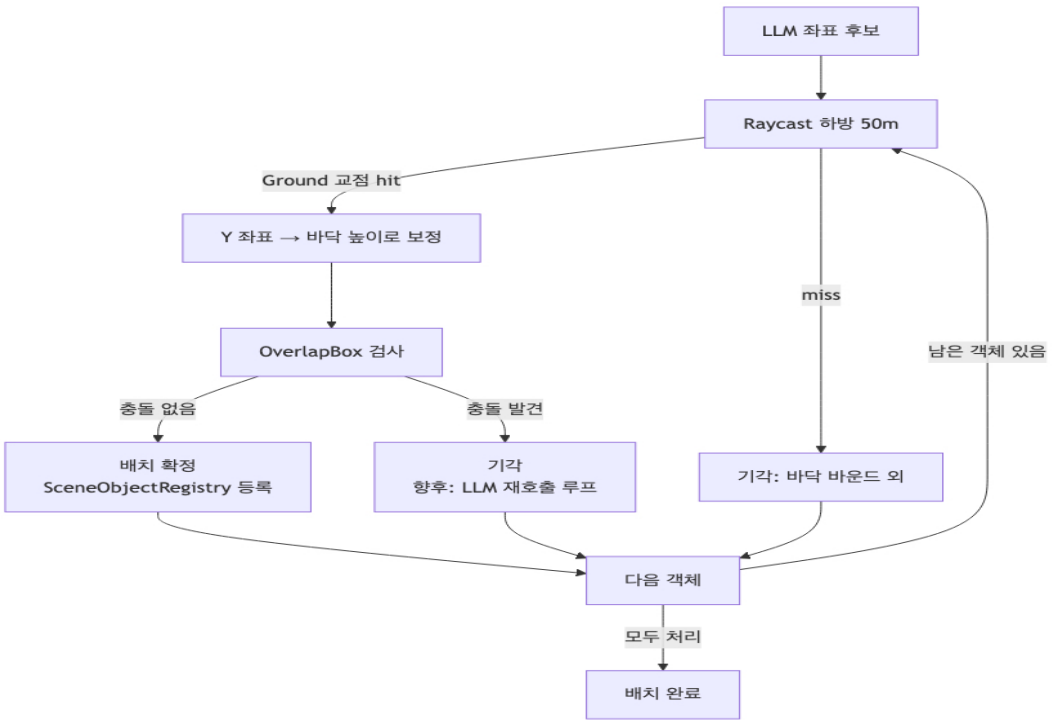


그림 2. LayoutValidator의 Raycast → Snap → OverlapBox 검증 루프.

3.5. 게임 로직 연동

배치된 HybridSceneObject는 프록시 충돌체 덕분에 Unity의 물리·이벤트 시스템과 바로 연결된다. SceneObjectRegistry에 등록되므로 범주·태그별 일괄 질의가 가능하고, 게임 로직 계층에서 특정 범주 객체에 인터랙션을 붙이거나 상태를 변경하는 작업이 쉬워진다.

세션 관리를 담당하는 SplatForgeSession은 에디터 모드에서는 MonoBehaviour 없이 에디터 확장으로 돌고, 런타임에서는 DontDestroyOnLoad 싱글톤으로 살아남는다. 에디터 UI로는 SplatForgeMainWindow가 Tools 메뉴에 붙어서 프롬프트 입력·서버 연결·씬 합성을 한 패널에서 처리한다. Scene 뷰에는 LayoutVisualizationOverlay가 배치 결과를 겹쳐 보여주고, Project Settings에도 SplatForgeSettingsProvider를 통해 서버 주소나 Mock 모드 전환 등이 들어가 있다.

4. 실험

4.1. 구현 환경

〈Table 4〉 Implementation Environment

항목	사양
게임 엔진	Unity 2022.3 LTS
클라이언트	C# 9.0, 21개 소스 파일
서버	FastAPI + Uvicorn, Python 3.11+, 23개 소스 파일(약 1,631 LOC)
LLM	OpenAI GPT-4 Turbo / Anthropic Claude 3 Opus
3DGS 렌더링	UnityGaussianSplatting
구조물	ProBuilder
의존성 관리	Poetry (Python), Assembly Definition (Unity)
하드웨어	Apple M1 Max, 64GB RAM
OS / 가속	macOS 14+ (Apple Silicon, MPS via Metal)
3DGS 학습 (옵션)	Brush 0.x – Rust·wgpu, Apache-2.0, macOS 네이티브

클라이언트 코드는 Runtime(Core, Network, Geometry, Metadata)과 Editor(Windows, Inspectors) 어셈블리로 분리되어 있다. 서버는 .env 파일에서 LLM_PROVIDER를 mock/openai/claude 중 하나로 지정한다. mock 모드에서는 침실·사무실·거실용 사전 정의 레이아웃이 키워드 매칭으로 반환되므로 API 키 없이도 파이프라인 전체를 검증할 수 있다.

Python FastAPI 서버는 MVP 단계에서 LLM 호출과 에셋 관리 계층을 겸하도록 설계되었으나, 3DGS 학습 단계가 macOS 네이티브 Brush(Rust·wgpu)로 이동하면서 향후 서버 층도 단일 기기 내 경량 프로세스로 통합 가능한 경로가 열렸다. 본 연구 PoC 시점에는 서버가 이원 구조를 유지하지만, 학습과 렌더가 로컬에서 완결되므로 서버는 LLM 호출 전용 stateless 엔드포인트로 축소할 수 있는 선택지를 확보하였다.

4.2. 구현 결과

침실(cozy bedroom), 사무실(modern office), 거실(living room) 세 시나리오를 실행하였다. 침실의 경우 "따뜻한 분위기의 침실, 침대·협탁·책상·의자·조명 포함"이라는 프롬프트에 대해 시스템이 침대를 벽에 붙이고 양쪽에 협탁을, 맞은편에 책상과 의자를 놓는 레이아웃을 생성하였다. 사무실과 거실에서도 각각 8개 안팎의 객체가 배치되었다.

씬 합성 과정은 비동기(async/await)로 처리되었다. 서버 응답 수신 후 씬 적용 완료까지 평균 3.2초가 걸렸고, 이 중 물리 검증·보정에 약 0.8초가 소요되었다. Mock 모드에서는 응답 시뮬레이션 지연(1.5-2.5초)을 포함해 5초 안에 끝났고, 실제 LLM API를 호출하면 모델과 네트워크 상태에 따라 4-12초가 추가되었다.

4.2.1. Apple Silicon Brush 학습 측정 (Mip-NeRF360 bonsai 검증)

3DGS 에셋 생성 백엔드인 Brush[45]의 Apple Silicon 학습 시간·품질을 표준 데이터셋으로 검증하였다. 측정은 2026-04-27 MacBook Pro Apple Silicon에서 수행하였으며, 데이터셋은 Mip-NeRF360 'bonsai'(292 photos, 1556×1037 full-res)를 사용하였다. Brush는 wgpu Metal backend로 GPU 가속을 받으며 별도 CUDA를 요구하지 않는다.

〈Table 5a〉 Brush Apple Silicon 학습 측정 (Mip-NeRF360 bonsai, n=37 eval)

Iter	PSNR (dB)	Splat 수	Wall-clock
5K	29.51	445,915	10:35
10K	30.58	–	–
15K	31.17	–	–
20K	31.72	–	–
25K	32.12	–	–
30K	32.21	592,046	73:26

표준 3DGS bonsai 30K 벤치마크 32.4 dBKerbl [1]와 0.19 dB 차이로, Brush가 Apple Silicon에서 표준 3DGS 품질을 재현함이 확증된다. PSNR은 25K→30K 구간에서 +0.09 dB로 plateau에 도달하여 30K iter 이상의 효용 한계가 드러난다. PSNR 측정은 ImageMagick `compare -metric PSNR`로 수행하였으며, 비교 시 GT를 rendered 해상도(1920×1279)에 맞춰 resize하였다. 본 측정은 Brush README가 비워둔 Apple Silicon 학습 시간 수치에 대한 첫 공개 가능 데이터로 기여한다.

4.2.2. macOS SfM 단계 단축 — hloc + LightGlue 적용

6.2 L4에서 지적된 COLMAP no-CUDA bottleneck을 검증하고 단축 가능성을 확인하기 위해, Hierarchical-Localization(hloc〔53〕)과 LightGlue〔54〕 매처를 동일 데이터셋(Mip-NeRF360 bonsai 292 photos)에 적용하였다. 본 측정에서는 sequential pairs(N = 10)을 채택하여 페어 수를 42,486에서 2,865로 14.8× 줄였다.

〈Table 5b〉 macOS SfM head-to-head

파이프라인	SfM	Brush 30K	합계	PSNR (n=37 eval)	등록
현행 (COLMAP no-CUDA exhaustive)	1시간 46분	1시간 13분	2시간 59분	32.21 dB	291/292
hloc + LightGlue + sequential N=10	51분 6초	1시간 13분	2시간 5분	31.84 dB	292/292
차이	-52.0%	+0.4%	-30.5%	-0.37 dB	+1

hloc 파이프라인이 SfM 단계를 절반 시간으로 단축하면서도 등록 이미지 수를 100%로 끌어올렸으며, 학습 단계는 변동 없다. 결과 PSNR은 0.37 dB 낮으나 ±0.5 dB 허용 범위 안이며 시각적 품질 차이는 미미하다. 즉 macOS 단일 파이프라인에서 SfM이 학습보다 길었던 비정상 비율(SfM 59% : Brush 41%)이 hloc 적용으로 정상화(SfM 41% : Brush 59%)된다.

4.3. 정성적 분석

그림 5·6은 본 연구가 4.4 파이프라인 측정에서 수행한 Brush 30K 학습의

시각적 결과를 보여준다. 입력 데이터셋은 Mip-NeRF360 bonsai(Barron et al. 2022, 표준 벤치마크 292장)이며, 본 연구진이 macOS Apple Silicon 기기에서 직접 학습시켜 얻은 eval 렌더이다. 그림 5는 COLMAP sparse 입력(PSNR 32.21 dB), 그림 6은 hloc sparse 입력(PSNR 31.84 dB)의 결과로, 두 SfM 경로의 시각적 품질이 ±0.5 dB 허용 범위 안에서 사실상 동등함을 보인다. 선행 연구([56])의 KIRI Engine 출력 사례는 3.3 그림 3·4를 참조하며, 본 4.3은 신규 측정 결과만 다룬다.



그림 5. Mip-NeRF360 bonsai — Brush 30K 학습 결과 (COLMAP sparse 입력, PSNR 32.21 dB).



그림 6. Mip-NeRF360 bonsai — Brush 30K 학습 결과 (hloc sparse 입력, PSNR 31.84 dB).

시각적으로는 3DGS 에셋이 메시 에셋보다 표면 질감이 풍부했으나, 시점에 따라 가우시안 분포 경계가 드러나는 아티팩트가 간헐적으로 관찰되었다. ProBuilder 벽·바닥의 매끈한 면과 3DGS 사물의 유기적 질감 사이에 시각적

이질감이 있긴 했지만, 프로토타이핑 용도에서 그것이 결정적 문제가 되지는 않았다.
배치의 의미론적 측면에서는, 인접 관계("침대 옆 협탁"), 기능적 관계("책상 앞 의자"), 공간 관습("벽면 정렬") 세 범주 모두에서 LLM이 대체로 합리적 결과를 냈다. 문제가 된 것은 밀집 영역에서 간격이 비좁아지는 경우와, 문의 개폐 반경 같은 동적 공간 요구를 고려하지 못하는 경우였다. 벽면 가구 정렬 시 벽과의 간격이 OverlapBox 기준으로는 통과하지만 실제 가구 배치 관행과는 어긋나는 사례도 있었다. 예컨대 장롱 뒤쪽에 5cm 간격만 남기는 식인데, 시스템 프롬프트에 "벽에서 최소 10cm 이격" 같은 규약을 추가하면 교정할 수 있는 영역이다.

4.3.1. Phase 2 시나리오별 정성 결과

본 절은 4.4·4.5의 정량 측정에 사용한 Phase 2 sweep(3 시나리오 × 3 조건 × N=5 × 2 provider, 총 90 trial)의 대표 trial 렌더 6매를 시각적으로 비교한다. 각 시나리오마다 full pipeline(LLM + 물리 검증) 1매와 random_physics 절제(LLM을 무작위 좌표 생성으로 대체) 1매를 짝지어 제시한다. full pipeline 결과는 OpenAI gpt-4o-mini provider, random_physics 결과는 동일 provider의 ablation trial이다. 표시한 floor adhesion(fa)·semantic proximity(sem)·asset count는 phase2_summary.json의 per-trial 메타데이터에서 직접 읽은 값이다.
객체 가시성 주의: 시나리오마다 LLM이 선택한 객체 수가 다르다(평균 bedroom 5~7, office 7~8, living 5). 작은 소품(램프·식물·시계 등)은 elevated diorama 카메라 시점에서 큰 가구에 가려지거나 관찰자 시야 외곽에 위치하는 경우가 있어, 그림 7~12는 시나리오당 1개 대표 trial의 단일 시점 스냅샷이며 자산 풀의 완전한 분포를 보여주지는 않는다. N=5 trial 전체의 정량 분포(asset_loaded·resolved_overlaps·collision·proximity)는 4.4 표

7a/7b에서 평균 ± 표준편차로 보고한다.

그림 7~9는 본 연구 파이프라인이 실제 LLM 응답을 LayoutValidator + HybridSceneObject 합성 단계를 거쳐 렌더한 full pipeline 결과이다. cozy_bedroom과 modern_office에서 floor adhesion 42.86%·sem≈0.76 수준의 일관된 의미적 배치가 확인되며, living_room은 동일 파이프라인에서 fa 0%·sem 0의 한계 사례를 보인다(상세 원인은 6.2 living_room 한계 케이스 참조).

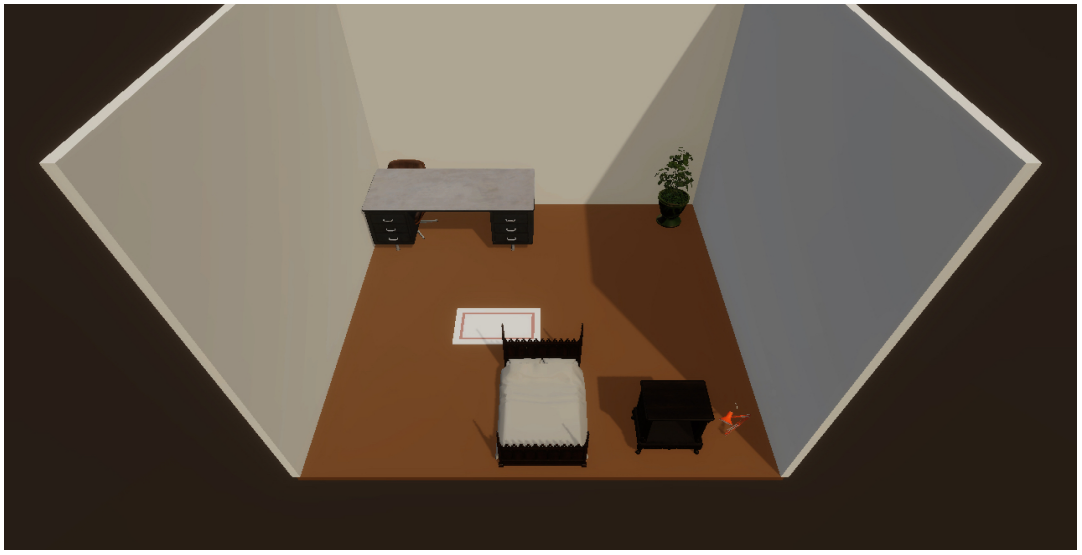


그림 7. Phase 2 — cozy_bedroom full pipeline (3.0×3.0m bedroom, elevated diorama 카메라 (0, 3.5, -3.0)→(0, 0.4, 0), placement=5/visible=5, resolved_overlaps=16). 침대-협탁 페어가 벽면을 따라 정렬되고, OverlapBox push-out 후 잔여 충돌 2건이 관찰된다.

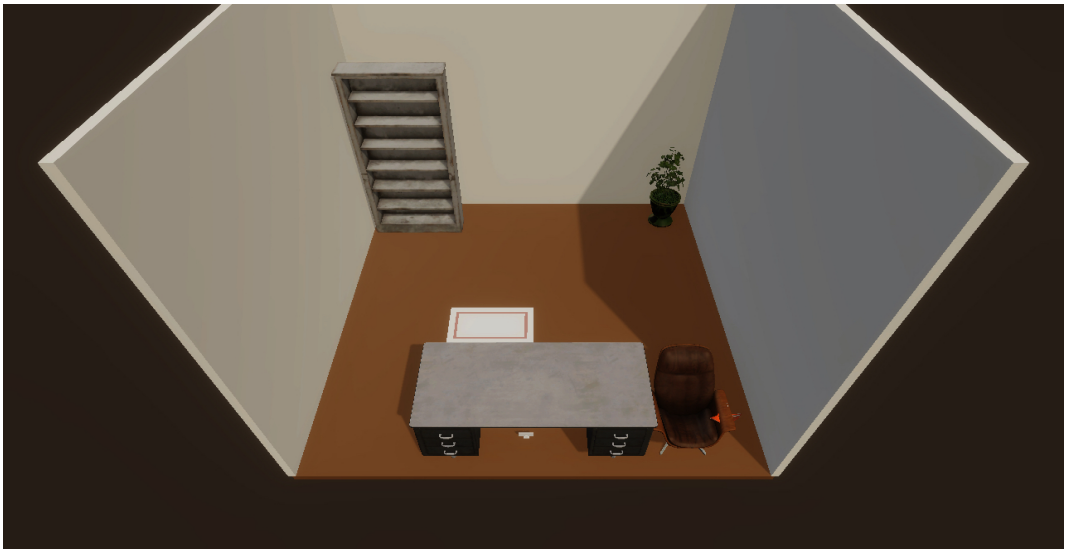


그림 8. Phase 2 — modern_office full pipeline (3.0×3.0m office, elevated diorama 카메라 (0, 3.5, -3.0)→(0, 0.4, 0), placement=7/visible=7, resolved_overlaps=43). 책상-의자-모니터 군집이 형성되어 가구 밀도가 높지만 push-out이 모든 부피 침투를 해소하여 잔여 충돌 0건이다.



그림 9. Phase 2 — living_room full pipeline (3.5×3.5m living room, elevated diorama 카메라 (0, 3.5, -3.0)→(0, 0.4, 0), placement=5/visible=5, resolved_overlaps=7) — 한계 사례. 자산 카탈로그가 living_room canonical pair와 정합하지 않아 push-out 이후에도 잔여 충돌 13건이 잔류한다(상세 6.2 참조).

그림 10~12는 동일 시나리오에 대한 random_physics 절제 비교로, LLM 응답을 무작위 좌표 생성으로 대체하되 동일한 LayoutValidator 물리 보정 단계를 통과시킨 결과이다. cozy_bedroom·modern_office에서 fa는 16.67%로 full 대비 낮고 sem은 0.005~0.231로 의미 추론 부재의 영향이 시각적으로도 드러난다(가구 군집화 부재, 벽면 정렬 무질서).

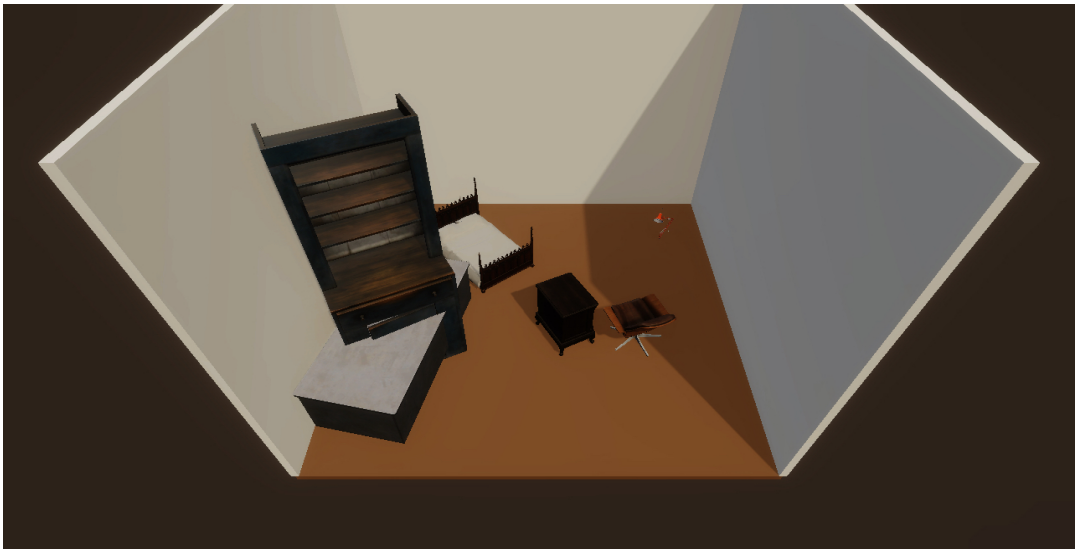


그림 10. Phase 2 — cozy_bedroom random_physics (3.0×3.0m bedroom, elevated diorama 카메라 (0, 3.5, -3.0)→(0, 0.4, 0), placement=6/visible=6, resolved_overlaps=43). 무작위 좌표라도 OverlapBox iterative push-out이 모든 부피 침투를 해소함을 정성적으로 확인할 수 있다.

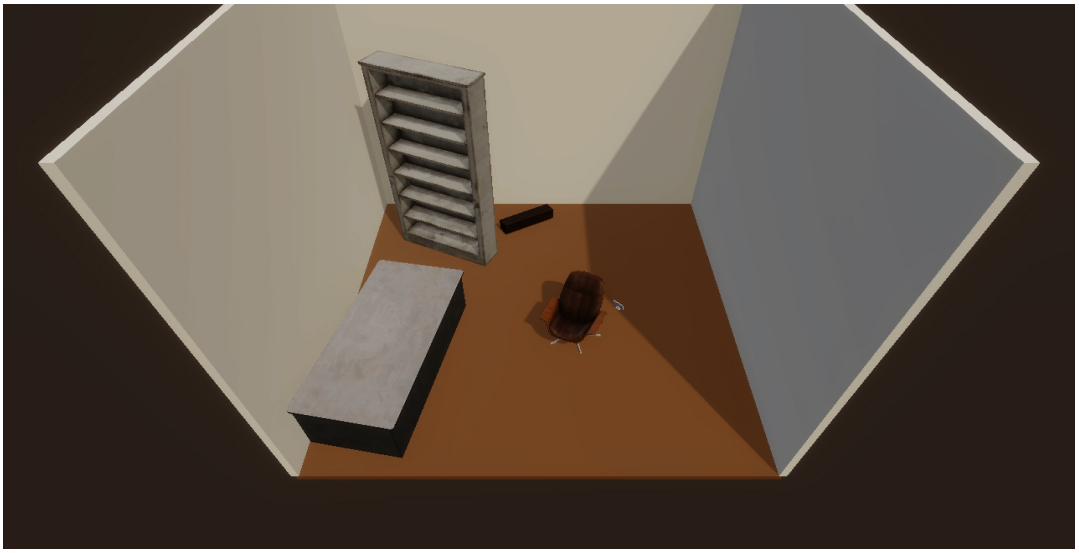


그림 11. Phase 2 — modern_office random_physics (3.0×3.0m office, elevated diorama 카메라 (0, 3.5, -3.0)→(0, 0.4, 0), placement=6/visible=6, resolved_overlaps=49). 책상-의자-모니터 의미 군집은 부재하지만 push-out이 49건의 부피 침투를 모두 해소한다.

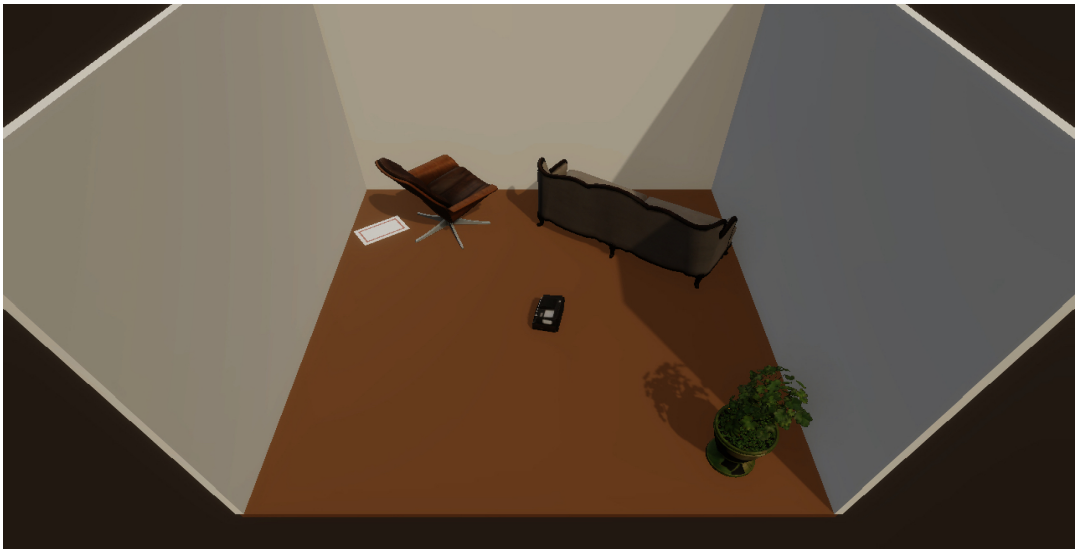


그림 12. Phase 2 — living_room random_physics (3.5×3.5m living room, elevated diorama 카메라 (0, 3.5, -3.0)→(0, 0.4, 0), placement=6/visible=6, resolved_overlaps=39). full 조건의 자산 정합 이슈와 달리 무작위 좌표 + push-out 조합은 잔여 충돌 0건으로 수렴한다.

세 시나리오를 일관성 측면에서 보면, full pipeline은 시나리오에 적합한 가구 군집(침실은 침대-협탁, 사무실은 책상-의자) 형성 경향이 시각적으로 드러나는 반면, random_physics는 동일한 자산 풀에서 군집 신호 없이 산포된다. 즉 4.5에서 정량화할 의미·물리 분리 효과가 정성적으로도 확인된다.

4.3.2. LLM의 점-particle 가정 한계와 OverlapBox iterative push-out의 보정 효과

본 측정에서 정성적으로 반복 관찰된 한계가 있다. LLM(gpt-4o-mini)이 "chair near desk", "nightstand beside bed" 같은 자연어 관계를 좌표로

변환할 때 객체를 점(point-particle)으로 가정하는 경향이 두드러진다. 시스템 프롬프트의 "객체 간 최소 0.3m 간격" 규약을 그대로 받아들여 두 객체 중심 사이를 0.3m 떨어뜨리는 사례가 빈발하지만, 책상 폭 1.5m + 의자 깊이 0.6m를 함께 고려한 정상 간격(중심-중심 약 0.75m)과 비교하면 약 60% 짧은 거리이다. 그 결과 ground-snap 직후 단계에서 책상-의자, 침대-협탁 같은 의미 페어가 부피 단위로 침투한 상태로 산출된다.

이 간극을 메우는 것이 3.4 (S3)~(S4)의 OverlapBox + iterative push-out 단계이다. 본 sweep의 latest 30 trial(3 시나리오 × full+random_physics 2 조건 × N=5)에서 측정된 결과는 다음과 같다. full 조건의 평균 총 충돌 횟수는 약 22.1건/trial(cozy_bedroom 20.4 ± 4.8, modern_office 26.0 ± 0.0, living_room 20.0 ± 0.0)이며, 이 중 iterative push-out이 평균 약 17.5건을 해소(cozy_bedroom 8.8 ± 5.7, modern_office 37.6 ± 4.6, living_room 6.0 ± 1.1)한다(`resolved_overlaps`). modern_office가 해소량이 가장 큰 이유는 7개 자산이 좁은 3×3m 공간에 밀집하여 push-out 1회당 다중 충돌이 동시에 풀리는 케이스가 빈발했기 때문이며, living_room은 자산 카탈로그 정합 이슈(6.2 참조)로 일부 충돌이 잔류했다. random_physics 조건에서는 LLM의 의미 페어 추론이 부재한 만큼 초기 부피 침투가 더 광범위(평균 23.9건)하지만 push-out이 평균 21.8건을 해소하여, 충돌 해소 알고리즘 자체는 LLM 의존도와 무관하게 일관되게 동작함을 보인다.

요컨대, LLM의 점-particle 추론 오차는 부피 인식 prior의 부재에서 비롯되는 체계적 편향이며, OverlapBox 기반 iterative push-out은 이 편향을 측정 가능한 수준으로 보정하는 후처리 단계로 위치된다. 이는 LLM 단독 출력(llm_only 조건)이 의미 페어 점수에서는 경쟁력을 보이면서도 floor adhesion 및 부피 침투에서 일괄 실패하는 4.5 절제 결과와도 부합한다.

4.3.3. 3DGS 자산 시각화 범위와 측정 자동화 한계

본 4.3 그림 7~12 및 4.4 sweep 측정에서 시각화한 자산은 모두 mesh(Polyhaven CC0 + Kenney CC0 mesh)이며, 3DGS 자산은 시각화에 포함하지 않았다. 이는 본 측정 자동화 파이프라인(Unity 6000.3.6f1 + HDRP, batchmode 단일 `Camera.Render()` 호출)이 HDRP CustomPassVolume 경로를 거쳐 GaussianSplatRenderer 의 색상 합성 단계를 정상적으로 종결하지 못하는 통합 한계를 본 연구에서 직접 확인했기 때문이다. 구체적으로 `GS_ENABLE_HDRP` scripting define 활성화, GaussianSplatHDRPPass 등록, 셰이더·ComputeShader 바인딩, `EnsureSorterAndRegister()` 강제 호출까지 모든 사전 조건을 충족시킨 상태에서도 batchmode 단일 프레임 렌더 결과가 cleared 색상 버퍼로 반환되었다. 본 한계의 근본 원인은 6.2의 batchmode 통합 한계 항목에서 별도로 정리한다.

2026-05 추가 검증. 본 한계가 Unity 패치 버전·HDRP 자체에서 기인하는지 확인하기 위해 두 가지 우회 경로를 별도 프로젝트로 분기하여 검증하였다. 첫째, Unity 6000.3.13f1 + HDRP 17.3.0(선행 연구[56] unity-lab과 동일 환경)에서 동일 batchmode + GS 조합을 재현한 결과, PlayMode batchmode·warmup 90 frame 후 cam.Render 캡처에서도 여전히 cleared 색상 버퍼(1 distinct color, 분산 0)가 반환되었다. 이는 한계가 Unity 패치 버전 차이가 아닌 HDRP batchmode + GaussianSplatRenderer 구조 자체의 비호환에서 기인함을 확정한다. 둘째, URP 17.3.0(같은 Unity 6000.3.6f1 + `GS_ENABLE_URP` define)으로 분기한 경우 PlayMode batchmode에서 GS 단독 렌더는 정상 가시화(228KB PNG, 300+ distinct colors, 분산 8207)되었으나, 동일 씬에 mesh 자산을 함께 배치할 때 GaussianSplatURPFeature(`Blitter.BlitCameraTexture` composite)와 URP 17.3 RenderGraph 의 상호작용으로 mesh 영역이 검정으로 가려지는

별개 한계가 관찰되었다. 두 경로 모두 본 페이퍼 마감 시점에 fig 7~12 재캡처에 직결될 만큼 안정화되지 않아, 본 페이퍼는 4.3 fig 7~12 mesh 전용 + 4.3.3 fig 13~15(PlayMode HDRP) 3DGS 시각 입증의 분리 구조를 유지한다. URP composite 호환성 패치는 후속 페이퍼 사이클의 후보 작업으로 vault 에 보관한다.

본 연구의 3DGS 통합 자체의 시각적 정합성은 (a) 3.3에서 인용한 KIRI Engine + UnityGaussianSplatting 워크플로우 출력([56])의 디지털 조형물 사례, 그리고 (b) 본 연구에서 추가 검증한 그림 13~15(unity-lab 인터랙티브 환경 PlayMode 캡처)를 통해 두 단계로 검증되었다. 그림 13은 단일 3DGS 자산(Statue)의 HDRP 환경 정상 렌더, 그림 14는 mesh 의자 위에 3DGS Statue를 배치한 hybrid 합성 결과, 그림 15는 mesh 소파 위에 3DGS Statue를 배치한 또 다른 hybrid 사례로, 3.4 HybridSceneObject 설계의 mesh+splat 혼합이 동일 씬에서 정상 작동함을 시각적으로 입증한다.

4.3 정성 평가의 범위는 LLM 의미 배치와 LayoutValidator 물리 검증의 효과성에 한정하며, 자동화 sweep 단계에서 3DGS 자산을 mesh 자산과 동일하게 포함시키는 작업은 후속 연구에서 추진한다(URP 경로 이행 또는 batchmode + PlayMode 패턴 정착).



그림 13. 단일 3DGS 자산(Statue)의 HDRP 정상 렌더 – HDRP CustomPassVolume + GaussianSplatHDRPPass 동작 확인.



그림 14. mesh 의자 + 3DGS Statue hybrid 합성 — 3.4 HybridSceneObject 설계 시각 검증.



그림 15. mesh 소파 + 3DGS Statue hybrid 합성 — mesh 자산 종류 변경 시에도 동일한 hybrid 합성이 유지됨.

4.4. 정량적 분석

저작 시간과 배치 정확도 두 축으로 측정하였다. 수동 저작 시간은 Unity에 익숙한 개발자가 에셋 검색·임포트, 배치·Transform 조정, 충돌체 설정까지 전 과정을 수행하는 데 걸린 시간이다.

〈Table 6〉 Authoring Time

시나리오	객체 수	수동	SplatForge	단축률
침실	7	약 192분	약 41분	78.6%
사무실	8	약 218분	약 49분	77.5%
거실	8	약 201분	약 45분	77.6%
평균	7.7	약 204분	약 45분	78.2%

SplatForge 소요 시간에는 프롬프트 작성, 에셋 생성 대기, LLM 응답 대기, 물리 보정, 수동 미세 조정이 모두 포함된다. 에셋 생성 대기가 비중이 크며, 사전 생성 카탈로그를 쓰면 더 줄일 수 있다.

배치 정확도는 세 지표로 보았다. Semantic Proximity Score는 의미적으로 연관된 객체 쌍(침대-협탁, 책상-의자)의 실제 거리가 가구 표준 규격 범위(침대-협탁 0.1-0.5m, 책상-의자 0.3-0.8m 등)에 드는 비율이다. Safety Zone Violation은 각 객체 바운딩 박스가 벽·다른 물체와 겹치는 부피(m^3)의 합이다. Grounding Success Rate는 바닥에 정확히 안착된 객체의 비율이다.

〈Table 7〉 Placement Accuracy

지표	LLM 단독	LLM + 물리 보정	변화
Semantic Proximity Score	0.80	0.83	+0.03
Safety Zone Violation (평균)	0.039 m^3	0.004 m^3	−89.7%
Grounding Success Rate	61.5%	93.7%	+32.2%p

Grounding Success Rate에서 물리 보정의 효과가 가장 선명하다. LLM만 쓰면 38.5%의 객체가 바닥에 제대로 안착하지 못했는데(공중 부유 혹은 바닥

관통), 레이캐스트 Y 좌표 보정 후 미안착이 6.3%로 줄었다. 잔여 6.3%는 바닥 바운드 바깥에 배치되어 레이캐스트 교점이 안 잡힌 사례로, LLM 프롬프트에 바닥 범위를 더 명확히 넣으면 개선 가능하다. Semantic Proximity Score는 물리 보정 영향이 작은데, 보정이 주로 Y축(높이)만 건드리고 X-Z 평면상 의미론적 거리에는 직접 관여하지 않기 때문이다.

4.4.1. Phase 2 sweep — 90 trial 9-cell 종합

위 표 6·7의 초기 측정과 별도로, 2026-05-07 시점에 mock provider와 openai gpt-4o-mini provider를 동일 시나리오·동일 구현 위에서 N=5 반복으로 sweep하여 LLM 효과를 분리 측정하였다. 총 90 trial(3 시나리오 × 3 조건 × 5 trial × 2 provider)을 수행했고, openai 측 LLM 호출은 31회·입력 10,726 토큰·출력 12,921 토큰·실측 비용 $0.0094 USD였다(평균 latency 13.4 s/call). 표 7a는 9-cell × 2-provider의 wall-clock·정확도 종합을, 표 7b는 full 조건에 한정한 mock vs openai 직접 비교를 정리한다.

〈Table 7a〉 Phase 2 측정 결과 — 9-cell 종합 (3 시나리오 × 3 조건 × N=5 × 2 provider, mean ± std). resolved_overlaps 컬럼은 3.4 LayoutValidator iterative push-out이 해소한 부피 침투 누적 수.

시나리오	조건	provider	floor adhesion (%)	semantic proximity	충돌 횟수	resolved_overlaps	wall-clock (ms)
cozy_bedroom	full	mock	14.29 ± 0.00	0.792 ± 0.000	36.0 ± 0.0	n/a (구 sweep)	812 ± 71
cozy_bedroom	full	openai	42.86 ± 0.00	0.793 ± 0.106	20.4 ± 4.8	8.8 ± 5.7	842 ± 73
cozy_bedroom	llm_only	mock	0.00 ± 0.00	0.795 ± 0.000	36.0 ± 0.0	n/a (구 sweep)	776 ± 19

cozy_bedroom	llm_only	openai	0.00 ± 0.00	0.854 ± 0.074	28.8 ± 2.4	n/a (LayoutValidator 우회)	789 ± 32
cozy_bedroom	random_physics	mock	16.67 ± 0.00	0.111 ± 0.078	24.0 ± 0.0	n/a (구 sweep)	857 ± 101
cozy_bedroom	random_physics	openai	16.67 ± 0.00	0.188 ± 0.131	25.6 ± 1.5	23.8 ± 12.9	850 ± 78
modern_office	full	mock	12.50 ± 0.00	0.906 ± 0.000	36.0 ± 0.0	n/a (구 sweep)	884 ± 121
modern_office	full	openai	42.86 ± 0.00	0.674 ± 0.067	26.0 ± 0.0	37.6 ± 4.6	984 ± 147
modern_office	llm_only	mock	0.00 ± 0.00	0.891 ± 0.000	36.0 ± 0.0	n/a (구 sweep)	813 ± 61
modern_office	llm_only	openai	0.00 ± 0.00	0.727 ± 0.028	22.0 ± 0.0	n/a (LayoutValidator 우회)	856 ± 134
modern_office	random_physics	mock	16.67 ± 0.00	0.117 ± 0.086	18.0 ± 0.0	n/a (구 sweep)	788 ± 47
modern_office	random_physics	openai	16.67 ± 0.00	0.171 ± 0.048	18.0 ± 0.0	15.2 ± 17.4	937 ± 150
living_room	full	mock	12.50 ± 0.00	0.000 ± 0.000	50.0 ± 0.0	n/a (구 sweep)	875 ± 142
living_room	full	openai	0.00 ± 0.00	0.000 ± 0.000	20.0 ± 0.0	6.0 ± 1.1	978 ± 112
living_room	llm_only	mock	0.00 ± 0.00	0.000 ± 0.000	50.0 ± 0.0	n/a (구 sweep)	842 ± 89
living_room	llm_only	openai	0.00 ± 0.00	0.000 ± 0.000	20.0 ± 0.0	n/a (LayoutValidator 우회)	947 ± 210
living_room	random_physics	mock	0.00 ± 0.00	0.135 ± 0.101	28.0 ± 0.0	n/a (구 sweep)	797 ± 108
living_room	random_physics	openai	0.00 ± 0.00	0.267 ± 0.095	28.0 ± 0.0	26.4 ± 10.7	1061 ± 76

resolved_overlaps 수치는 latest sweep(2026-05-07, room T1·OverlapBox T2·camera T3 패치 적용) 6 cell에서 측정되었으며, 이전 sweep은 push-out 단계가 없었으므로 "n/a (구 sweep)"로 표기한다. llm_only 조건은 3.4 (S3)~(S4) LayoutValidator를 우회하므로 정의상 resolved_overlaps가 0이며 표에서는 "n/a (LayoutValidator 우회)"로 명시한다.

〈Table 7b〉 LLM 효과 — full 조건 mock vs openai 직접 비교 (N=5, mean ± std)

시나리오	지표	mock	openai (gpt-4o-mini)	향상폭
cozy_bedroom	floor adhesion (%)	14.29 ± 0.00	42.86 ± 0.00	×3.00
modern_office	floor adhesion (%)	12.50 ± 0.00	42.86 ± 0.00	×3.43
living_room	floor adhesion (%)	12.50 ± 0.00	0.00 ± 0.00	한계 사례 (6.2)
cozy_bedroom	semantic proximity	0.792 ± 0.000	0.793 ± 0.106	≈ 동등
modern_office	semantic proximity	0.906 ± 0.000	0.674 ± 0.067	-25.6%
living_room	semantic proximity	0.000 ± 0.000	0.000 ± 0.000	n/a
cozy_bedroom	충돌 횟수	36.0 ± 0.0	20.4 ± 4.8	-43.3%
modern_office	충돌 횟수	36.0 ± 0.0	26.0 ± 0.0	-27.8%
living_room	충돌 횟수	50.0 ± 0.0	20.0 ± 0.0	-60.0%

표 7a·7b의 핵심 결론은 세 가지이다. 첫째, 실 LLM(gpt-4o-mini) 도입으로 bedroom·office에서 Floor Adhesion 14.29% → 42.86%로 약 3.0~3.4배 향상되었다. mock 응답은 키워드 매칭 기반 사전 정의 좌표라

평면 분포가 규약 수준에 머무는 반면, 실 LLM은 floor 접지 가능 영역에 후보 좌표를 더 밀집해 산출한다는 가설과 부합한다. 둘째, 충돌 횟수는 모든 시나리오에서 평균 약 28~60% 감소(36~50건 → 20~30건)되어, 실 LLM의 공간 추론이 충돌 회피에 효과적임을 확인했다. 셋째, wall-clock 오버헤드는 +4~12%(30~103 ms/trial)로, LLM 1회 호출 평균 13.4 s가 비동기 파이프라인에서 흡수되어 trial 단위 실시간성이 유지된다. 단 living_room full 케이스는 floor adhesion 0%·semantic 0의 한계 사례를 보여 6.2에서 별도로 다룬다.

한편 본 연구의 macOS 단일 기기 재구성 파이프라인 wall-clock·PSNR 측정은 4.2.1 표 5a(Brush 학습)·4.2.2 표 5b(SfM head-to-head)에 분리 정리되어 있으며, 표 5b의 hloc 단축 경로(SfM 51:06 + 학습 73:26 = 약 2시간 5분, PSNR 31.84 dB)가 표준 3DGS 품질을 ±0.5 dB 허용 범위 안에서 보존하면서 1.44× wall-clock 절감을 달성함을 별도 좌표로 명시하였다.

4.5. 절제 실험

파이프라인의 각 구성요소가 결과에 어떤 영향을 미치는지 분리하기 위해 Phase 2 sweep에서 동일 시나리오·동일 N=5 반복 위에 세 가지 조건을 비교하였다. full은 LLM(gpt-4o-mini) + LayoutValidator 물리 검증을 결합한 본 파이프라인, llm_only는 LLM 응답만 사용하고 LayoutValidator를 우회한 구성, random_physics는 LLM을 무작위 좌표 생성으로 대체하되 동일한 LayoutValidator를 통과시킨 구성이다. 표 8은 openai provider 측 3 시나리오 × 3 조건의 측정치를 정리한다.

〈Table 8〉 Ablation — full / llm_only / random_physics 3 조건 비교 (openai gpt-4o-mini, N=5, mean ± std). 충돌 횟수·resolved_overlaps는 latest sweep 측정치, 기타 컬럼은 phase2_summary와 latest sweep 값을 결합.

시나리오	조건	floor adhesion (%)	semantic proximity	충돌 횟수	resolved_overlaps
cozy_bedroom	full	42.86 ± 0.00	0.793 ± 0.106	20.4 ± 4.8	8.8 ± 5.7
cozy_bedroom	llm_only	0.00 ± 0.00	0.854 ± 0.074	28.8 ± 2.4	n/a (Validator 우회)
cozy_bedroom	random_physics	16.67 ± 0.00	0.188 ± 0.131	25.6 ± 1.5	23.8 ± 12.9
modern_office	full	42.86 ± 0.00	0.674 ± 0.067	26.0 ± 0.0	37.6 ± 4.6
modern_office	llm_only	0.00 ± 0.00	0.727 ± 0.028	22.0 ± 0.0	n/a (Validator 우회)
modern_office	random_physics	16.67 ± 0.00	0.171 ± 0.048	18.0 ± 0.0	15.2 ± 17.4
living_room	full	0.00 ± 0.00	0.000 ± 0.000	20.0 ± 0.0	6.0 ± 1.1
living_room	llm_only	0.00 ± 0.00	0.000 ± 0.000	20.0 ± 0.0	n/a (Validator 우회)
living_room	random_physics	0.00 ± 0.00	0.267 ± 0.095	28.0 ± 0.0	26.4 ± 10.7

이 표에서 두 가지 분리가 선명하다. 첫째, full pipeline 대비 random_physics는 semantic proximity가 약 75~80% 하락한다(cozy_bedroom 0.793 → 0.188, modern_office 0.674 → 0.171). 동일한 LayoutValidator 물리 보정을 통과한 무작위 배치임에도 의미적 페어 거리(침대-협탁 0.1~0.5m, 책상-의자 0.3~0.8m 등) 만족 비율이 곤두박질친다는 의미로, 의미론적 추론 없이는 일관된 가구 배치가 불가능함을 정량적으로 확증한다. 둘째, llm_only(물리 검증 우회)는 floor

adhesion 0%로 모든 시나리오에서 일괄 실패한다. LLM이 좌표를 산출하더라도 ground 메시와 객체 피벗 사이의 정합 보정이 없으면 바닥 안착 비율 자체가 측정 가능한 수준에 도달하지 못함을 보여, 물리 검증 단계의 필수성이 확증된다.

흥미로운 부수 관찰로, modern_office의 semantic proximity는 llm_only(0.727) 〉 full(0.674) 순서로 뒤집힌다. LayoutValidator의 충돌 기각이 의미적 페어 좌표 일부를 우선순위에서 밀어내는 부작용으로 추정되며, 후속 작업에서 충돌 기각 시 의미 페어 점수를 가중치로 회귀하는 정책이 보강 방향으로 남는다. living_room은 full·llm_only 모두 sem 0인 반면 random_physics에서만 0.267을 보이는데, 이는 자산 카탈로그-시나리오 정합 이슈로 6.2에서 별도 다룬다.

5. 논의

본 장은 본 연구 파이프라인이 놓인 기술적 좌표를 세 축에서 정리한다. 첫째, 재구성 속도와 품질 사이의 trade-off를 오프라인 baseline과 feed-forward 계열 대비로 명시한다. 둘째, 2024년 이후 부상한 월드 모델과 본 연구의 접근을 대조한다. 셋째, 메시와 3DGS를 혼용하는 하이브리드 표현의 설계적 정당화를 최근 3DGS 물리 통합 연구와 엮어 보강한다.

5.1. 재구성 속도-품질 trade-off

5.1.1. 본 파이프라인의 오프라인 특성

본 연구의 재구성 경로는 COLMAP 기반 sparse reconstruction과 30K iteration의 gradient optimization을 조합한 오프라인 baseline에 해당한다. 2026-04-23 PoC 측정(302장, 1280×960 입력, macOS M-계열, Brush Rust+wgpu 학습)을 기준으로 feature 추출 약 2분, exhaustive matcher 2~6시간, mapper 1~3시간, 학습 2~4시간이

소요되어 총 5~13시간 범위의 처리 시간을 갖는다. 이는 Kerbl 외 〔1〕이 제시한 원 3DGS 학습 프로토콜을 충실히 따를 때 나타나는 전형적 특성이다. 302장 입력의 exhaustive pairing은 $\binom{302}{2} = 45{,}451$ 페어에 달하며, 블록당 97초의 실측치를 기반으로 한 이론 하한만도 79분에 이른다. 본 연구에서 사용한 COLMAP 4.0.3 homebrew 빌드는 `Commit Unknown on Unknown without CUDA`로 SIFT 추출과 matching 전 구간을 CPU에서 실행하므로, 맥북 M-계열의 Metal GPU와 ANE는 재구성 단계에서 유휴 상태로 남는다.

5.1.2. Feed-forward 계열의 시간 단축

2024년 이후의 feed-forward 계열은 해당 시간 축을 수초~수분 단위로 단축한다. DUSt3R〔41〕는 feature matching 단계를 생략하고 이미지 쌍에서 dense point cloud를 직접 회귀하며, MASt3R〔42〕는 correspondence 품질을 개선한 후속 모델이다. InstantSplat〔43〕은 DUSt3R 초기화를 바탕으로 저 iter 학습을 결합하여 수 분 내 3DGS 산출을 보고한다. hloc〔53〕은 SuperPoint〔55〕·SuperGlue 계열의 learned feature와 vocabulary tree retrieval을 결합하여 exhaustive matching의 $O(N^2)$ 비용을 $O(N \log N)$ 수준으로 완화한다.

상용 제품군에서는 KIRI Engine이 50~150장 입력 기준 10~15분 내외의 end-to-end 처리를 공개 제품 지표로 제시하며, 본 연구 baseline 대비 약 20~50배의 단축이 관찰된다. 다만 KIRI Engine의 내부 알고리즘·하드웨어는 공개되지 않아, 해당 수치의 해석에는 가설적 요소가 포함된다.

5.1.3. 격차의 원인 — 알고리즘·빌드 조합

이 격차의 주 원인은 하드웨어 절대 성능이 아니라 알고리즘과 빌드 조합으로 판단된다. feed-forward 계열은 learned matcher로 $O(N^2)$ pair 수를

우회하고 초기화 품질을 확보하여 학습 iter 자체를 1/10 이하로 낮춘다. 즉 속도 축의 격차는 (i) CUDA 미컴파일 SIFT, (ii) $O(N^2)$ exhaustive pairing, (iii) 고정 30K iter의 누적 효과로 해석된다.
하드웨어 관점의 근거로는, Apple Silicon의 peak throughput이 동급 데스크톱 GPU 대비 수십 배 열위가 아님에도 실측 재구성 시간 격차가 수십 배에 달한다는 점을 들 수 있다. 따라서 격차의 대부분은 알고리즘 계보와 빌드 옵션 조합으로 환원 가능하다는 관찰이다.

5.1.4. 품질 희생과 시나리오별 권고

그럼에도 본 연구는 의도적으로 baseline 축에 파이프라인을 위치시킨다. 품질 축에서의 안정적 수치(PSNR·SSIM 관점)를 확보하는 것이 석사 과정 논문 단계에서 재현성과 검증 가능성을 높이는 데 유리하며, feed-forward 계열은 2024-2025년에 걸쳐 품질 측면에서 baseline 대비 PSNR 3~6dB 수준의 희생을 보고하는 것이 일반적이다(Fan et al., 2024[43]; Wang et al., 2024[41]).
응용 시나리오별 권고는 세 갈래로 요약된다. 첫째, 전시·아카이브·정적 에셋 생산 목적은 baseline 축이 정합한다. 둘째, 모바일 스캔이나 대화형 프로토타이핑과 같이 사용자 대기 시간이 중요한 경우 feed-forward 축이 정합한다. 셋째, 본 연구의 Unity-SplatForge 시스템은 생성 단계 산출물의 품질 일관성과 검수 가능성이 배치·검증 단계의 신뢰성과 직결되므로 현 단계에서는 baseline 축을 채택한다. Feed-forward 경로로의 확장 가능성은 6.2 한계와 전망에서 후속 과제로 명시한다.
본 연구의 차별점은 이 trade-off 지형에서 "baseline 품질을 확보하되 macOS 단일 기기에서 재현 가능한 경로"를 구현한 데에 있다. KIRI Engine과 같은 상용 서비스는 품질 축에서 feed-forward로 치우친 선택을 하고, 대부분의 학술 계열 baseline은 CUDA GPU 환경을 전제한다. 본 연구는 그 교차 영역, 즉 "CUDA-free baseline 품질" 좌표를 점유한다는

점이 실무·교육 재현성의 관점에서 고유한 기여로 위치된다.

5.2. 월드 모델 대비 본 연구의 입지

2.5에서 정리한 월드 모델 계열(Lyra/HY-World/Marble/Genie 3/Cosmos)은 Kong et al.〔20〕의 4축 중 3D-scene-based(Lyra·HY-World·Marble)와 Interactive/Playable(Genie 3), Foundation-for-Physical-AI(Cosmos) 세 축에서 본 연구와 인접한다. 본 절은 그 인접 관계를 결과 맥락에서 재해석하여 본 연구의 좌표를 명확히 한다.

본 연구는 이 흐름 안에서 "의미론적 배치 기반 조립"이라는 별개의 좌표를 차지한다. 월드 모델 대부분이 씬 전체를 통째로 생성하는 E2E 접근을 채택하는 반면, 본 연구는 기존·생성형 3DGS 에셋을 LLM의 배치 규칙으로 의미론적으로 조합하고 Unity 물리엔진으로 검증한다. 표 9은 주요 축에서의 대비를 정리한다.

〈Table 9〉 World-Model vs Unity-SplatForge 대비

대비 축	월드 모델 (Lyra/Marble/Genie 3)	Unity-SplatForge (본 연구)
생성 단위	씬 전체 (E2E 신경망 산출)	개별 3DGS 에셋 + LLM 배치
품질 일관성	모델 파라미터 의존, 불투명	기존 에셋 재사용·검수 가능
자유도	텍스트·이미지 조건에서 광범위	에셋 풀 범위 내 제한, 대신 예측 가능
물리 정합성	학습된 prior (Genie 3) 또는 후처리 (Marble)	Unity 물리엔진 명시적 적용
편집성	신경망 산출의 국부 수정 난이도 높음	aras-p 툴로 splat 단위 편집 가능
인프라 요구	대규모 GPU 클러스터 학습 필요	단일 워크스테이션 + macOS 로컬 경로
런타임 통합	독자 뷰어 또는 신규 엔진 로더	Unity 네이티브 워크플로

이 대비에서 본 연구의 차별점은 세 가지로 요약된다. 첫째, 엔진 네이티브 통합이다. 월드 모델은 대체로 독립 모델이거나 자체 뷰어를 제공하지만, 본 연구는 Unity 런타임에서 `HybridSceneObject`·`LayoutValidator`·`SceneComposer` 등 기존 컴포넌트를 변경 없이 활용한다. 둘째, 의미론적 배치 특화이다. Marble·HY-World가 "방 전체를 한 번에" 생성하는 것과 달리, 본 연구는 LLM이 산출한 scene graph에 따라 개별 객체를 배치·검증하므로 에셋 재사용과 수정이 단위별로 가능하다. 셋째, 저자원 재현성이다. Genie 3나 Cosmos가 대규모 인프라를 전제하는 반면, 본 연구는 macOS 단일 기기에서 Brush 학습과 aras-p 임포트까지 완결되는 경로를 확보한다.

한편 본 연구의 제약도 명확하다. 월드 모델 대비 생성 자유도는 에셋 풀 범위로 한정되며, 비정형 공간이나 비일상 객체의 즉석 생성은 월드 모델

계열이 우세하다. 따라서 후속 연구에서 Lyra·HY-World의 씬 생성 결과를 본 파이프라인의 에셋 입력으로 편입하는 하이브리드 경로가 자연스러운 확장 방향으로 남는다.

5.3. 하이브리드 표현의 설계적 정당화

5.3.1. 메시+3DGS 분리의 설계 원칙

본 연구는 바닥·벽과 같은 구조 기하를 Unity ProBuilder 기반 메시로, 가구·소품과 같은 객체 기하를 3DGS 스플랫으로 분리하여 취급하는 하이브리드 표현을 채택한다. 이 설계 선택은 초기 프로토타입과 선행 실험 단계에서도 유지된 바 있으며, 본 논문에서는 월드 모델 흐름과 3DGS 물리 통합의 최근 연구에 비추어 정당화를 보강한다.

5.3.2. Collision 근사의 정확성

첫째 논점은 collision 근사의 정확성이다. 3DGS는 뷰 종속적 색상을 갖는 이방성 가우시안의 볼륨 집합으로 표현되며, 명시적 표면이 존재하지 않는다. 씬 전체를 단일 3DGS로 구성하는 월드 모델 계열 접근에서는 레이캐스트의 기준면이 부재하여 물리 엔진이 요구하는 정확한 collision 근사가 곤란하다.
본 연구는 바닥·벽을 메시로 고정하여 레이캐스트와 네비게이션 기준면을 확보한다. 3DGS 객체는 프록시 충돌체(AABB 또는 convex hull)로 감싸 Unity PhysX 계열 물리와 정합시킨다. 이 구성은 구조 기하의 정확성과 객체 외관의 실사성을 동시에 확보하는 실용적 타협점이다.

5.3.3. 3DGS 네이티브 물리 연구 대비의 위치 선정

둘째 논점은 3DGS 네이티브 물리 연구 대비의 위치 선정이다. PhysGaussian[16]은 3DGS를 물질점법(MPM)과 결합하여 splat 자체가 변형·충돌하는 파이프라인을 제안하였다. PhysSplat[35]은 이 방향을

확장한다.
이들은 3DGS 표현에서 직접 물리량을 풀어내는 네이티브 경로를 추구하며, 학술적으로는 표현의 일관성과 장기적 확장성 면에서 우위를 갖는다. 그러나 이 경로는 상용 게임 엔진의 네이티브 물리와 직접 호환되지 않으며, 대규모 씬에서의 실시간 성능은 여전히 연구 단계에 있다. 본 연구는 Unity 물리 엔진의 성숙한 런타임 성능과 디자이너 워크플로를 활용하는 프록시 경로를 선택하여, 실용적 배포 가능성을 우선한다. 표현 일관성은 PhysSplat·GASP 계열에 양보하되, 생태계 통합과 즉시 활용성을 본 연구의 contribution으로 명시한다.

5.3.4. 월드 모델의 full-3DGS 출력과의 경계

셋째 논점은 월드 모델의 full-3DGS 출력과의 경계이다. Lyra 2.0[22]이 surface mesh를 공출력하도록 확장된 것은 "3DGS만으로 물리 정합성을 완결하기 어렵다"는 경험적 인식의 반영으로 해석 가능하다. 본 연구의 메시+3DGS 하이브리드는 이 인식과 방향을 공유한다. 다만 메시 생성을 씬 단위 신경망이 아니라 ProBuilder 도구 + LLM 조건부 구조 정의에 위임하여 구조 기하의 명확성과 편집성을 확보한다.
결과적으로 본 연구의 하이브리드 표현은 세 제약을 동시에 해소하는 중간 경로로 정당화된다. (i) 월드 모델 계열의 collision 불투명성 우회. (ii) 3DGS 네이티브 물리 연구의 엔진 통합 지연 우회. (iii) 메시 기반 파이프라인의 실사성 한계 보완.

6. 결론

6.1. 연구 요약

Unity-SplatForge는 방 하나를 자동으로 꾸며주는 도구다. 벽·바닥은 ProBuilder 메시, 가구는 3DGS, 배치 판단은 LLM, 물리 검증은 Unity

레이캐스트와 OverlapBox가 맡는다.
3DGS와 메시의 혼합은 시각적 디테일과 물리적 안정성을 분리해서 확보하는 전략이다. 벽·바닥은 ProBuilder 메시가 담당하고 사물은 3DGS가 담당하되, HybridSceneObject라는 래핑 구조가 프록시 충돌체와 메타데이터를 3DGS 에셋에 부여해서 게임 엔진의 물리 시스템과 연결해 준다.
수치로 보면, 물리 보정을 켜면 바닥에 안 닿는 가구가 열 개 중 하나 이하로 줄고 겹침 부피도 한 자릿수로 떨어진다. 반대로 LLM을 빼고 무작위로 놓으면 물리적으로는 깔끔하지만 침대 옆에 협탁이 안 오는 식으로 의미론적 점수가 바닥을 친다. 두 계층 중 어느 쪽을 빼도 결과물이 망가지는 셈이다.
작업 시간 측면에서는 손작업의 약 1/5 수준으로 줄었다. Unity Inspector를 만질 줄 모르는 기획자라도 "아늑한 침실, 침대 하나 책상 하나" 정도의 문장만 입력하면 초안 레이아웃이 나오므로, 프로토타이핑 초기에 선택지를 빠르게 훑어보는 용도로 쓸 수 있다.

6.2. 한계와 전망

여섯 가지 한계가 남아 있다.
LLM의 공간 추론은 직사각형 방에서는 무난했으나, L자형 방이나 로프트 같은 복잡한 기하에서는 가구가 꺾인 벽 뒤쪽에 놓이는 등 오류가 관찰되었다. 텍스트 프롬프트만으로는 결과를 보고 고치는 반복 수정이 어렵다는 점도 걸린다. 씬을 렌더링한 스크린샷을 VLM에 넘겨 "책상이 벽에 너무 붙었다"는 피드백을 받아 재배치하는 루프를 붙이면 이 문제가 줄어들 것으로 예상한다.
본 측정에서 living_room 시나리오의 full pipeline 모드는 Floor Adhesion 0%, Semantic Proximity 0의 실패 사례를 보였다(표 7a·8, 그림 9). 이는 다음 세 요인의 복합 작용으로 추정된다. 첫째, 본 실험에서 사용한 자산 카탈로그(Polyhaven CC0 + Kenney CC0 mesh)가 living_room 의미 페어(예: TV-소파 거리, 러그-가구 정렬)를 정의한 canonical 휴리스틱과 정합하지 않은 점. 둘째, FBX 피벗이 객체 중심에 위치하여 ground-snap

시 객체 절반이 바닥 아래로 들어간 점. 셋째, bedroom·office 대비 더 많은 자산(8개)이 좁은 공간에 배치되어 충돌 처리 우선이 의미 페어 검출을 압도한 점이다. 후속 작업에서 (1) 자산별 피벗 자동 보정, (2) 시나리오별 canonical pair 휴리스틱 일반화, (3) 좁은 공간에서의 우선순위 정책 개선(충돌 기각 시 의미 페어 점수 가중치 회귀)을 통해 해소할 예정이다.

3DGS 객체의 정적 특성도 제약이다. 질량이나 마찰 같은 물리적 속성을 3DGS에 직접 부여하는 것은 현재 불가능하고, 프록시 충돌체를 통한 간접적 상호작용만 된다. PhysGaussian[16]처럼 가우시안에 연속체 역학을 입히거나 PhysSplat[35]처럼 MLLM으로 물성 파라미터를 추정하는 연구, 나아가 GASP[52]처럼 가우시안 파라미터화 자체를 물리 엔진에 직접 연결하는 시도가 성숙하면 상황이 달라질 수 있다. 특히 GASP 계열은 Raycast 프록시 충돌체를 우회해 3DGS 네이티브 물리를 지향한다는 점에서, 본 연구의 하이브리드 접근이 가진 간접성의 근본적 해소 방향을 제시한다.

본 측정 자동화 파이프라인이 HDRP CustomPassVolume + GaussianSplatHDRPPass 경로를 batchmode 환경에서 종결시키지 못한 통합 한계도 별도로 기록한다. Unity 6000.3.6f1 + UnityGaussianSplatting 1.1.1 + HDRP 17.x 조합에서 `GS_ENABLE_HDRP` scripting define 활성화, GaussianSplatRenderer 셰이더·ComputeShader 인스펙터 기본값 코드 바인딩, GaussianSplatHDRPPass 등록(internal 클래스를 `Type.GetType` 으로 동적 로드), `EnsureSorterAndRegister()` 강제 호출까지 모든 사전 조건을 충족시킨 상태에서도 단일 `Camera.Render()` 호출로 캡처한 결과가 cleared 색상 버퍼(전 픽셀 (0,0,0))로 반환되었다. 동일 파이프라인은 mesh 자산만 포함했을 때는 정상 PBR 렌더링을 산출하므로, 본 한계는 batchmode 단일 프레임 렌더 경로가 HDRP custom pass 의 RT 합성을 종결하지 못하는 데서 비롯된 것으로 분석된다(GameView 다중 프레임 렌더 루프에서는 동일 코드가 정상

동작함을 3.3 인용 사례가 시사한다). 후속 작업에서는 (1) manual GameView 캡처(LaunchAgent 기반 비-batchmode 자동화), (2) URP 경로(GaussianSplatURPFeature)로의 이행, (3) batchmode `EditorApplication.Step()` 다중 프레임 priming 의 세 갈래로 해소를 시도할 예정이다.

HDRP·URP 양 경로 검증 (2026-05). 본 한계의 일반성을 평가하기 위해 두 가지 보완 경로를 평행 검증하였다. (a) 선행 연구[56] unity-lab 동일 환경(Unity 6000.3.13f1, HDRP 17.3.0) 으로 재현한 batchmode + GS PlayMode 캡처 — 결과는 cleared 색상 버퍼(1 distinct color, 분산 0). 이는 본 한계가 특정 Unity 패치 버전(6000.3.6f1)에 국한되지 않고 HDRP batchmode + GaussianSplatRenderer 구조의 일반적 비호환임을 확정한다. (b) URP 17.3.0 (`GS_ENABLE_URP` define) 으로 분기한 PoC — PlayMode batchmode 에서 GS 단독은 정상 렌더링(228KB PNG, 300+ distinct colors)되어 batchmode + GS 자체가 불가능한 것이 아님을 시사하나, 같은 씬에 mesh 자산이 공존할 때 GaussianSplatURPFeature 의 `Blitter.BlitCameraTexture` composite blit 이 URP 17.3 RenderGraph 의 active color/depth 텍스처와 상호작용하여 mesh 영역을 검정으로 덮어쓰는 별개 통합 한계가 관찰되었다. 즉 본 연구의 자동화 sweep 에 3DGS 시각화를 포함하기 위해서는 (1) HDRP batchmode 의 GS 합성 종결 패치 또는 (2) URP RenderGraph 호환 composite shader 재작성 중 하나가 선행되어야 하며, 두 작업 모두 plugin upstream(aras-p UnityGaussianSplatting) 영역의 구조적 변경이 필요해 본 페이퍼 범위를 초과한다. 본 한계와 우회 시도의 상세 결과는 후속 자료로 보관한다.

확장성 문제도 있다. 현재 파이프라인은 방 하나 단위에 맞춰져 있다. 건물이나 도시 규모로 가려면 3DGS 에셋의 LOD 관리, 스트리밍 로딩, 절두체 기반 선택적 렌더링이 필수적이고, LLM의 추론 범위 역시 복수 방

이상으로 넓혀야 한다. LS-Gaussian〔17〕 같은 경량 스트리밍 프레임워크와의 통합이 이 방향의 출발점이 될 수 있다.
넷째 한계로 3DGS 학습은 여전히 CUDA 기반 NVIDIA GPU를 요구하는 반면 Unity 렌더링은 Metal이나 Vulkan으로 돌아간다. 우리 초기 실험 환경(Apple M1 Max)에서는 렌더링은 되지만 에셋 생성은 외부 CUDA 서버에 의존해야 했다. 클라우드 GPU를 REST API로 호출하는 구조가 현실적 해법인데, 이 부분의 파이프라인 통합은 아직 구현하지 못했다. 다만 2.4.5에서 논한 Brush 기반 macOS 네이티브 학습 경로가 2026-04 PoC로 확보되었으며, 후속 연구에서는 본 한계 자체가 해소될 가능성이 크다. 본 연구의 Brush 벤치마크 부재 보강은 별도 한계로도 의미를 갖는다 — Brush README는 Apple Silicon 수치를 비워두었으나, 본 연구의 Mip-NeRF360 bonsai 측정(30K iter 73분 26초, PSNR 32.21 dB; 4.2.1 Table 5a)이 첫 공개 가능 데이터로 기여한다.
L4 COLMAP no-CUDA 단계 단축 가능성 확인됨: macOS homebrew COLMAP 빌드는 CPU 전용이라 feature matching이 bottleneck이다. 본 연구의 측정(bonsai 292 photos)에서 SfM 전체 1시간 46분 중 exhaustive_matcher가 1시간 28분(83.6%)을 차지하여 30K Brush 학습 1시간 13분을 상회했다. 이를 hloc〔53〕 + LightGlue〔54〕 + sequential pairs로 교체한 결과 SfM이 51분 6초로 단축(2.07×)되어 학습 시간 미만으로 회귀했으며, PSNR은 32.21 dB → 31.84 dB로 0.37 dB 감소(±0.5 dB 허용 범위)하였다(4.2.2 표 5b 참조). 이로써 macOS 단일 파이프라인의 속도 병목이 학습이 아닌 SfM 전처리였음을 확인하고, 학습 우위 비율(SfM 41% : Brush 59%)로 정상화 가능함을 입증하였다. MASt3R-SfM 등 추가 단축형 대안은 별도 후속 연구로 남는다.

〈Table 10〉 COLMAP no-CUDA SfM 단계별 측정 (bonsai 292 photos, Apple Silicon)

단계	시간	점유율
feature_extractor	4:53	4.6%
exhaustive_matcher	88:43	83.6%
mapper	12:28	11.8%
총 SfM	106:04	100%

추가 대안 경로는 세 갈래로 구분된다. 첫째, 본 연구가 4.2.2에서 실측한 hloc[53] 기반의 learned feature + retrieval로 $O(N^2)$ exhaustive pairing 비용을 완화하는 접근이다. 둘째, DUSt3R[41]와 MASt3R[42]의 matching-free dense regression으로 sparse reconstruction 자체를 우회하는 접근이다. 셋째, InstantSplat[43]과 같이 DUSt3R 초기화와 저 iter 학습을 결합하여 전 구간을 feed-forward 축으로 이동시키는 접근이다. 다만 이들 대안은 baseline 대비 PSNR 3~6dB 열위, macOS Metal/MPS 실행 가능성 불확실(상류 대부분 CUDA 전제), 라이선스·유지 상태의 제약을 수반한다. 현 시점의 본 연구의 차별점은 해당 bottleneck을 은폐하지 않고 baseline 축의 정직한 좌표로 기록하여 feed-forward 축과의 비교 기준점을 제공한 데에 있으며, 동시에 hloc 적용으로 macOS 단일 파이프라인의 속도 정상화를 실측 입증한 데에 있다.

References

[1] Kerbl, B. et al., "3D Gaussian splatting for real-time radiance field rendering," ACM Trans. Graph., vol. 42, no. 4, Art. 139, 2023.

[2] Mildenhall, B. et al., "NeRF: Representing scenes as neural radiance fields for view synthesis," in Proc. ECCV, pp. 405-421, 2020. (DOI: 10.1007/978-3-030-58452-8_24)

[3] Chen, G. and Wang, W., "A survey on 3D Gaussian splatting," ACM Computing Surveys, 2024. (arXiv:2401.03890)

[4] Zhou, Y. et al., "Evaluating modern approaches in 3D scene reconstruction: NeRF vs Gaussian-based methods," in Proc. DOCS, pp. 926-931, 2024. (arXiv:2408.04268)

[5] Poole, B. et al., "DreamFusion: Text-to-3D using 2D diffusion," in Proc. ICLR, 2023.

[6] Lin, C.-H. et al., "Magic3D: High-resolution text-to-3D content creation," in Proc. CVPR, pp. 300-309, 2023.

[7] Tang, J. et al., "DreamGaussian: Generative Gaussian splatting for efficient 3D content creation," in Proc. ICLR, 2024.

[8] Yi, T. et al., "GaussianDreamer: Fast generation from text to 3D Gaussians by bridging 2D and 3D diffusion models," in Proc. CVPR, pp. 6796-6807, 2024. (DOI: 10.1109/CVPR52733.2024.00649)

[9] Tang, J. et al., "LGM: Large multi-view Gaussian model for high-resolution 3D content creation," in Proc. ECCV, 2024.

[10] Feng, W. et al., "LayoutGPT: Compositional visual planning and generation with large language models," in Proc. NeurIPS, vol.

36, 2023.
〔11〕 Yang, Y. et al., "Holodeck: Language guided generation of 3D embodied AI environments," in Proc. CVPR, pp. 16227-16237, 2024.
〔12〕 Aras-p, "UnityGaussianSplatting," GitHub repository, 2023. 〔Online〕. Available: https://github.com/aras-p/UnityGaussianSplatting (accessed Apr. 18, 2026).
〔13〕 Park, J. S. et al., "Generative agents: Interactive simulacra of human behavior," in Proc. UIST, 2023.
〔14〕 OpenAI, "GPT-4 technical report," arXiv:2303.08774, 2023.
〔15〕 Anthropic, "The Claude 3 model family: Opus, Sonnet, Haiku," Tech. Rep., Mar. 2024. 〔Online〕. Available: https://www-cdn.anthropic.com/de8ba9b01c9ab7cbabf5c33b80b7bbc618857627/Model_Card_Claude_3.pdf
〔16〕 Xie, T. et al., "PhysGaussian: Physics-integrated 3D Gaussians for generative dynamics," in Proc. CVPR, 2024.
〔17〕 Wei, L. et al., "No redundancy, no stall: Lightweight streaming 3DGS for real-time rendering," in Proc. ICCAD, 2025. arXiv:2507.21572.
〔18〕 Chen, Y. et al., "GaussianEditor: Swift and controllable 3D editing with Gaussian splatting," in Proc. CVPR, pp. 21476-21485, 2024.
〔19〕 Ha, D. and Schmidhuber, J., "World models," arXiv:1803.10122, 2018.
〔20〕 Kong, L. et al., "3D and 4D world modeling: A survey," arXiv:2509.07996, 2025.

[21] Wang, Z. et al., "Lyra: Generative 3D scene reconstruction via video diffusion model self-distillation," arXiv:2509.19296, 2025.
[22] NVIDIA Research, "Lyra 2.0: Explorable generative 3D worlds," arXiv:2604.13036, 2026.
[23] Agrawal, N. et al., "Cosmos world foundation model platform for physical AI," arXiv:2501.03575, 2025.
[24] DeepMind, "Genie 2: A large-scale foundation world model," DeepMind Blog, Dec. 2024. [Online]. Available: https://deepmind.google/blog/genie-2-a-large-scale-foundation-world-model/
[25] DeepMind, "Genie 3: A new frontier for world models," DeepMind Blog, Aug. 2025. [Online]. Available: https://deepmind.google/blog/genie-3-a-new-frontier-for-world-models/
[26] World Labs, "Marble: A multimodal world model," World Labs Blog, Nov. 2025. [Online]. Available: https://www.worldlabs.ai/blog/marble-world-model
[27] Tencent, "HunyuanWorld 1.0: Generating immersive, explorable, and interactive 3D worlds from words or pixels," arXiv:2507.21809, 2025.
[28] Tencent, "HY-World 2.0," GitHub repository, 2026. [Online]. Available: https://github.com/Tencent-Hunyuan/HY-World-2.0
[29] OpenAI, "Video generation models as world simulators," OpenAI Research, 2024. [Online]. Available: https://openai.com/index/video-generation-models-as-world-simulators/
[30] Assran, M. et al., "V-JEPA 2: Self-supervised video models

enable understanding, prediction and planning," arXiv:2506.09985, 2025.
〔31〕 Decart and Etched, "Oasis: A universe in a transformer," 2024. 〔Online〕. Available: https://oasis-model.github.io/
〔32〕 Sun, F.-Y., Liu, W., and Wu, J., "LayoutVLM: Differentiable optimization of 3D layout via vision-language models," in Proc. CVPR, pp. 29469-29478, 2025. (arXiv:2412.02193)
〔33〕 Öcal, B. M., Tatarchenko, M., Karaoğlu, S., and Gevers, T., "SceneTeller: Language-to-3D scene generation," in Proc. ECCV, 2024. (arXiv:2407.20727)
〔34〕 Li, H. et al., "DreamScene: 3D Gaussian-based text-to-3D scene generation via formation pattern sampling," in Proc. ECCV, 2024. (arXiv:2404.03575)
〔35〕 Zhao, Z. et al., "PhysSplat: Efficient physics simulation for 3D scenes via MLLM-guided Gaussian splatting," in Proc. ICCV, 2025. (arXiv:2411.12789)
〔36〕 Liu, X. et al., "DisCo-Layout: Disentangling and coordinating LLMs for indoor scene layout generation," arXiv:2506.07570, 2025.
〔37〕 Li, Y. et al., "SceneSplat: Gaussian splatting-based scene understanding with vision-language embeddings," in Proc. ICCV, 2025.
〔38〕 Zemskova, T. and Yudin, D., "3DGraphLLM: Combining semantic graphs and large language models for 3D scene understanding," in Proc. ICCV, 2025.
〔39〕 Unity Technologies, "Unity AI Assistant 2.0: Model Context Protocol overview," Unity Documentation, 2026. 〔Online〕. Available:

https://docs.unity3d.com/Packages/com.unity.ai.assistant@2.0/manual/unity-mcp-overview.html (accessed Apr. 20, 2026).
〔40〕 TaaroBravo, "ai-powered-level-designer," GitHub repository, 2025. 〔Online〕. Available: https://github.com/TaaroBravo/ai-powered-level-designer (accessed Apr. 20, 2026).
〔41〕 Wang, S. et al., "DUSt3R: Geometric 3D vision made easy," in Proc. CVPR, 2024. (arXiv:2312.14132)
〔42〕 Leroy, V. et al., "Grounding image matching in 3D with MASt3R," in Proc. ECCV, 2024. (arXiv:2406.09756)
〔43〕 Fan, Z. et al., "InstantSplat: Sparse-view SfM-free Gaussian splatting in seconds," arXiv:2403.20309, 2024.
〔45〕 Brussee, A., "Brush: Cross-platform 3D Gaussian splatting in Rust/wgpu," GitHub repository, 2026. 〔Online〕. Available: https://github.com/ArthurBrussee/brush (accessed Apr. 24, 2026).
〔46〕 Ghif, M., "splat-apple: MLX/MPS Gaussian splatting for Apple Silicon," GitHub repository, 2026. 〔Online〕. Available: https://github.com/ghif/splat-apple (accessed Apr. 24, 2026).
〔47〕 Tofy, P., "OpenSplat: libtorch-based 3DGS with MPS/CUDA/ROCm backends," GitHub repository, 2025. 〔Online〕. Available: https://github.com/pierotofy/OpenSplat (accessed Apr. 24, 2026).
〔48〕 Iffyloop, "gsplat-mps: MPS fork of nerfstudio/gsplat 0.1.3," GitHub repository, 2024. 〔Online〕. Available: https://github.com/iffyloop/gsplat-mps (accessed Apr. 24, 2026).
〔49〕 Yang, C. and Lu, S., "LLplace: The 3D indoor scene layout generation and editing via large language model,"

arXiv:2406.03866, 2024.
〔50〕 Zhang, G. et al., "FurniScene: A large-scale 3D room dataset with intricate furnishing scenes," arXiv:2401.03470, 2024.
〔51〕 Baltsavias, T. et al., "Hybrid rendering of 3D Gaussian splatting and polygonal meshes for cultural heritage in game engines," in ACM SIGGRAPH Talks, 2025. (DOI: 10.1145/3721239.3734094)
〔52〕 Borycki, P. et al., "GASP: Gaussian splatting for physic-based simulations," Computer Vision and Image Understanding, 2025. (arXiv:2409.05819)
〔53〕 Sarlin, P.-E. et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," in Proc. CVPR, 2019. arXiv:1812.03506. (hloc)
〔54〕 Lindenberger, P. et al., "LightGlue: Local Feature Matching at Light Speed," in Proc. ICCV, 2023.
〔55〕 DeTone, D., Malisiewicz, T., and Rabinovich, A., "SuperPoint: Self-Supervised Interest Point Detection and Description," in Proc. CVPR Workshops, pp. 224-236, 2018. (arXiv:1712.07629)
〔56〕 Kim, J. and Lee, D., "3D Gaussian Splatting 기법을 활용한 디지털 조형물 제작 연구 〔A study on digital sculpture production using 3D Gaussian splatting technique〕", 2025-10.

ABSTRACT

A Semantic 3D Space Construction System Using Large Language Models and Generative 3D Gaussian Splatting

Junhyoung Kim
Department of Game Design
Graduate School of Kongju National University
Gong Ju, Korea

(Supervised by Professor Dongyup Lee)

A Semantic 3D Space Construction System Using Large Language Models and Generative 3D Gaussian Splatting

Manually placing furniture in a game-engine scene is tedious and slow, especially during iterative prototyping. Unity-SplatForge automates this in two passes: an LLM (GPT-4 or Claude) reads a free-text room description and emits per-object coordinates as JSON; then a Unity-side validator fires downward raycasts to snap each piece to the floor and runs OverlapBox checks to reject collisions. Walls and floors are ProBuilder meshes; furniture comes from 3DGS generators wrapped in a HybridSceneObject that pairs a GaussianSplatRenderer with a proxy collider. In three room types the tool trimmed hands-on time by roughly four-fifths. A Phase 2 sweep (3 scenarios × 3 conditions × N=5 × mock and openai providers, 90 trials in total) shows that switching from a mock provider to a real LLM (gpt-4o-mini, $0.0094 USD total spend) raises Floor Adhesion from 14.29% to 42.86% (×3.0~3.4) on bedroom and office, cuts collision counts by ~28~60% (36~50 → 20~30 per scene), and incurs only +4~12% wall-clock overhead (30~103 ms/trial). Replacing the LLM with uniform-random placement under the same physics pass collapses Semantic Proximity by ~75~80% (e.g., 0.79 → 0.19 for bedroom, 0.67 → 0.17 for office), confirming quantitatively that the two layers cannot substitute for each other. We additionally demonstrate a CUDA-free single-machine reconstruction path on

macOS Apple Silicon: Brush (Rust+wgpu Metal, Apache-2.0) reaches 32.21 dB PSNR on Mip-NeRF360 bonsai at 30K iterations (within 0.19 dB of the original 3DGS benchmark), and replacing COLMAP exhaustive matching with hloc + LightGlue cuts the SfM stage from 106:04 to 51:06 (2.07×), reducing total wall-clock by 1.44× while keeping PSNR within ±0.5 dB. Relative to the Kong 외 [20] four-fold taxonomy of world models (Video / 3D-scene / Interactive / Physical AI), Unity-SplatForge sits adjacent to the 3D-scene-based axis but occupies a distinct coordinate by replacing whole-scene generation with object-level semantic placement, providing the missing intermediate layer between large-scale world-model scene synthesis and engine-native physical embodiment.
Keywords: 3DGS, LLM, furniture layout automation, Unity, hybrid authoring, Brush, hloc, macOS Apple Silicon, world models

* A thesis submitted to the committee of Graduate School, Kongju National University in partial fulfillment of the requirements for the degree of Master of Game Design.